OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual	**Name: M.Hasmath Nisha M**
Duration: 1 Week	**Reg no:192524513**
Total Marks: 50	
Submission Date: _____	

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        //   advance time, reduce left[p]; add every process that has now arrived and is
        //   not yet added to the queue; then put p back at the end of the queue if it
        //   still has time left, otherwise record finish[p] and count it as done

    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    return 0;
}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process TAT WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
        //   closed the pipe); keep the count, the sum and the maximum

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors — 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
        //   circularly, unlock, then post full

    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
        //   out circularly, unlock, post empty and add the item to consumed_sum

    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm — 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```c
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
            //   finish: add alloc[p] to work, mark finish[p], append p to seq and set
            //   found = 1

        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

> **Model → Design → Implement → Measure → Compare → Evaluate → Recommend**

Assignment 2: Processes, Scheduling, Threads, IPC, Synchronisation and Deadlock

Summary of Decisions

Every decision below is defended in its own section and rests on a program I wrote and ran, not on textbook values alone.

Question	Decision	Key evidence
Q1 Exam server (6)	Fixed pool of worker processes, with a separate lower-priority pool for proctoring video	Starting 2,000 processes took 207 ms (small parent) to 1,576 ms (64 MB parent); a thread costs about 9 KB resident
Q2 Video service (8)	Multilevel feedback queue (quanta 2, 6, then FCFS) with ageing	Average wait 9.2 against 18.6 for FCFS; thumbnails wait 2.0 against 23.0; 8 context switches against 25 for Round Robin q=1
Q3 Plate reader (6)	Pool of threads sized to the cores, a few more if frames wait for I/O	Speed-up 1.9 to 2.1 on 2 cores for any pool size; 2.8 to 3.0 when frames also wait 10 ms
Q4 Weather pipeline (6)	Pipe for the sensor-to-analyser link; shared memory only above about 1 GB/s	100 MB: pipe 2.6 GB/s, shared memory 10.1 GB/s; readings are only 16 bytes each
Q5 Ticket booking (11)	One lock per coach, taken in fixed train-then-coach order	3.1 times the global lock at 4 threads (2.97 against 0.96 million bookings/s); no double sales
Q6 Cluster deadlock (13)	Banker's algorithm at job admission, resource ordering for ad-hoc locks, detection every few minutes as a backup	Safety check 7.9 µs typical and 2.2 ms worst case at 1,000 jobs; grows as n squared

Environment. All programs were compiled with gcc 13.3 (-O2) and run on Linux 6.18 in a virtual machine with 2 CPUs (Intel Xeon 2.8 GHz). Times from a virtual machine vary by up to about 30% between runs, so I report medians or ranges where it matters. Numbers marked "typical" or "assumed" are standard values or stated assumptions, not measurements.

Given data. The problem sheet did not include the code skeletons, the job table for Q2 or the resource state for Q6. I wrote the programs myself and chose the data: a five-job video workload for Q2, and the standard five-process, three-resource Banker's state (A = 10, B = 5, C = 7) for Q6. Both are shown in full so they can be replaced.

Q1. Processes and Process States (6 marks)

Conclusion. A separate process per student is sound for isolation but wasteful at 2,000 sessions. I recommend a fixed pool of worker processes for sessions and a separate, lower-priority pool for proctoring video, with answers saved to a database so a worker crash loses nothing.

1.1 Process-state diagram

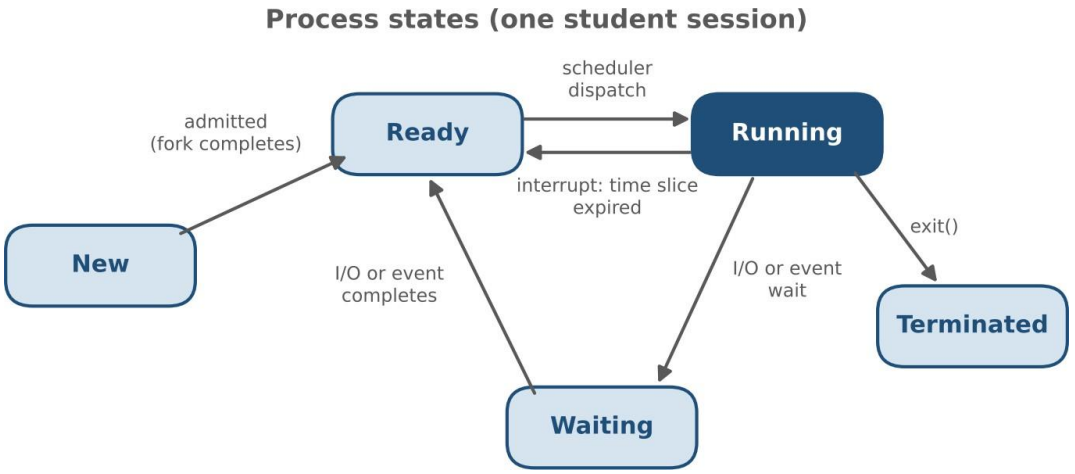


Figure 1. Process states and the events that move a session between them

Transition	Cause	In the exam server
New to Ready	The process has been created and admitted	The server forks a session process when a student logs in
Ready to Running	The scheduler dispatches it to a CPU	A session gets a CPU to process an answer
Running to Ready	Timer interrupt: its time slice ended	A busy session is pre-empted so others can run
Running to Waiting	It requests I/O or waits for an event	Waits for the database, the network or the next click
Waiting to Ready	The I/O finishes or the event happens	The answer was saved; the student sends the next request
Running to Terminated	exit() is called	Student submits; session ends

1.2 Process control block (PCB)

The PCB is the kernel's record of one process; it is what the scheduler saves and restores at a context switch. In Linux it is the task_struct.

Process Control Block (PCB) of one process

Process ID (PID), parent PID, user ID	identity
Process state	new / ready / running / waiting / terminated
Program counter	address of the next instruction
CPU registers, stack pointer	saved at every context switch
Scheduling information	priority, queue pointers, CPU time used
Memory-management information	page-table pointer, base / limit
Accounting information	CPU time, start time, limits
I/O status information	open files (descriptor table), devices held

Figure 2. Fields of a process control block

1.3 Program: creating children with fork() and observing them

forkdemo.c (Appendix A) forks three children. Each child sleeps for 4 seconds, which puts it in the waiting state, and then exits with status 10, 11 or 12. The parent reaps them with wait(). While they slept I ran ps and read /proc:

```
$ ps -o pid,ppid,stat,wchan:14,comm --ppid <parent>
  PID  PPID STAT WCHAN          COMMAND
10556 10554 S    hrtimer_nanosl forkdemo
10557 10554 S    hrtimer_nanosl forkdemo
10558 10554 S    hrtimer_nanosl forkdemo

$ grep -E 'State|Pid|PPid|VmRSS|ctxt' /proc/10556/status
State:  S (sleeping)
Pid:    10556
PPid:   10554
VmRSS:      928 kB
voluntary_ctxt_switches:     1
nonvoluntary_ctxt_switches:  0

parent pid=10554
child 0: pid=10556 ppid=10554
child 1: pid=10557 ppid=10554
child 2: pid=10558 ppid=10554
parent: child 10556 finished, status 10
parent: child 10557 finished, status 11
parent: child 10558 finished, status 12
```

All three children show PPID 10554 (the parent), state S and the kernel function hrtimer_nanosleep they are blocked in. S is Linux's name for the textbook waiting state; the single voluntary context switch is the moment the child gave up the CPU to sleep. Mapping of Linux letters to the five states:

Linux STAT	Textbook state
R	Running (on a CPU) or Ready (in the run queue); ps does not separate them
S	Waiting (interruptible sleep, for example a timer or socket)
D	Waiting (uninterruptible, usually disk I/O)
Z	Terminated, waiting for the parent to call wait() (a zombie)

1.4 Evaluation: where the time goes with 2,000 active processes

I measured the three costs the question names. many.c forks 2,000 sleeping processes and reads their memory from /proc; ctx.c measures a context switch by passing one byte between two processes over pipes, both pinned to one CPU.

Cost	Measured	Meaning for 2,000 sessions
Creating 2,000 processes, small parent	207 ms (104 µs each)	A login burst is a serial queue in the parent: sessions 1 and 2,000 start 0.2 s apart
Creating 2,000 processes, parent holding 64 MB	1,576 ms (788 µs each)	fork copies page tables, so the cost grows with the parent's size
Memory per process, small parent	RSS 302 kB, proportional set (PSS) 28 kB, page tables 48 kB	About 76 kB of unshared memory per idle process (PSS + page tables), so about 150 MB for 2,000, plus a kernel stack of about 16 kB each (typical)
Memory per process, 64 MB parent	RSS 65.8 MB, PSS 60 kB, page tables 180 kB	RSS is misleading: copy-on-write shares the 64 MB until a process writes to it
One context switch (process to process)	1.9, 2.4 and 2.4 µs in three runs	Includes two pipe system calls, so the switch itself is under 2 µs

Context-switch cost. If all 2,000 sessions were runnable on one core and each got a slice of about 1 ms (typical), one full round would cost 2,000 × 2 µs = 4 ms of switching, which is 0.2% of the 2-second round. Direct switching cost is therefore small. The real damage is indirect: each switch leaves the next process with a cold cache and TLB, and 2,000 working sets cannot all stay cached.

Memory. The measured overhead is small, but a real session process runs an application runtime. With an assumed 30 MB of private memory per session, 2,000 sessions need 60 GB; the same sessions as threads share one runtime. When memory runs out the machine swaps, and swapping is the most likely cause of sessions that "hang".

Waiting for I/O. Most session time is spent waiting (for the network, the database, the next keystroke), which is why 2,000 processes can share a few cores. An illustrative workload model with assumed figures on a 16-core server:

Activity (assumed)	Rate	CPU per event	Cores needed
Session requests	2,000 sessions × 0.5 per s	2 ms	2
Auto-save	2,000 sessions × 0.1 per s	5 ms	1
Proctoring video	2,000 streams × 1 per s	20 ms	40
Total			**43 cores against 16 available**

Under this model proctoring needs most of the CPU and starves the sessions, so they appear to hang even though the session code is cheap. The design fix is to separate proctoring from sessions and give it lower priority, not to change how sessions are created.

Judgement. Process per student gives excellent isolation (a crash or memory error stays inside one session, and the kernel enforces each student's privacy) and is simple to write. It is sound at hundreds of sessions. At 2,000 it costs a login-time fork queue, hundreds of megabytes of overhead, and 2,000 competing working sets, and it does not scale to 20,000.

1.5 Design decision

Criterion	Process per student	Thread per student	Fixed pool of worker processes
Start-up cost	104 to 788 µs per session (measured)	52 µs per thread (measured earlier)	Paid once at server start; a login costs a queue entry
Memory per session	28 kB PSS + 48 kB page tables + runtime	About 9 kB resident per thread (10.3 MB for 1,001 threads, measured)	A few kB of session state in a worker; runtime shared by the sessions in that worker
Crash isolation	Best: one session	Worst: one fault kills all sessions	One worker: about 1/16 of sessions
Context switches	Many, between 2,000 processes	Fewer address-space switches	Fewest: one per core, work multiplexed by an event loop
Control of resources	Per-process limits and priorities	Weak	Per-pool limits: proctoring pool can be niced
Scaling to 20,000	Poor	Good	Good

Decision: a fixed pool of worker processes (about one to two per core, for example 16), each handling many sessions with an event loop or a small thread pool, plus a separate pool for proctoring and a queue for auto-saves. Answers go to the database every few seconds, so a worker crash loses at most a few seconds of one session's work, and a supervisor restarts the worker. This keeps most of the isolation (16 failure domains instead of one), removes the login-time fork queue, and cuts memory to the pool size.

Trade-off accepted. A bug in one worker affects about 125 sessions (2,000 divided by 16). Raising the pool to 64 workers cuts that to about 31 sessions at the price of more memory and switching.

1.6 Alternative: each session as a thread in one server process

Measure	Thread per session	Process per session
Context-switch time	1.8, 2.5 and 1.6 µs per switch (measured, same pipe test); no significant difference from processes on one CPU. In theory a thread switch is cheaper because the address space and TLB are kept	1.9, 2.4 and 2.4 µs
Creation time	52 µs per thread (from my earlier measurement on this machine)	183 µs with a small parent; 788 µs with 64 MB
Memory per session	A stack of a few KB to MB of virtual space; measured about 9 KB resident per thread with 64 KiB stacks (10.3 MB for 1,001 threads)	A full address space: page tables 48 kB plus kernel stack and structures, plus the runtime
One session crashes	A segmentation fault or memory corruption in one thread kills the whole process: all 2,000 sessions are lost	Only that session ends

Threads win on memory and start-up cost, but the crash behaviour is unacceptable for an examination without a supervisor and frequent auto-save. That is why the final design uses a pool of processes, each of which may use threads internally.

Q2. CPU Scheduling (8 marks)

Conclusion. I recommend a multilevel feedback queue with ageing. It gives thumbnails an average wait of 2.0 time units instead of 23.0 under FCFS, needs no advance knowledge of job length, and costs O(1) per decision.

2.1 The job set

Times are in abstract units (a thumbnail is 1 to 2, a film is 20). Priority 1 is highest. The same data is built into the C program.

Job	Type	Arrival	Burst	Priority
J1	film	0	20	3
J2	thumbnail	1	2	1
J3	clip	2	8	4
J4	thumbnail	3	1	2
J5	clip	4	5	5

Definitions used: turnaround = completion time − arrival; waiting = turnaround − burst; response = first time on the CPU − arrival. Priority is non-preemptive; SRTF is the preemptive form of SJF. Round Robin admits jobs that arrive during a slice before the pre-empted job rejoins the queue. A context switch is counted whenever the CPU moves from one job to a different one.

2.2 Gantt charts, waiting, turnaround and response times

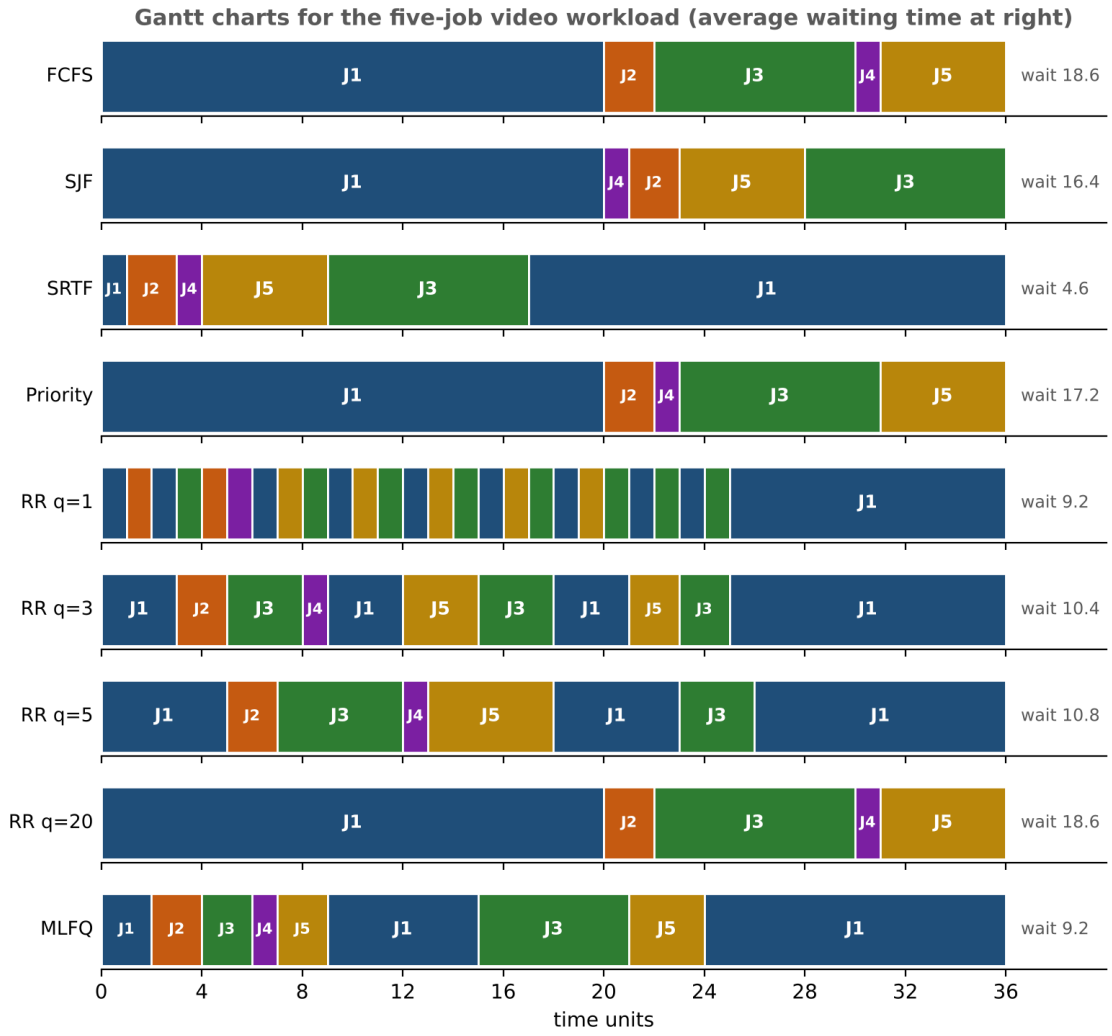


Figure 3. Gantt charts for every algorithm on the same five jobs

Per-job results (waiting / turnaround / response). Hand check for FCFS: J2 arrives at 1 and starts at 20, so waiting = 19, response = 19 and turnaround = 22 − 1 = 21.

Job	FCFS	SJF	SRTF	Priority	Round Robin q=3
J1 film	0 / 20 / 0	0 / 20 / 0	16 / 36 / 0	0 / 20 / 0	16 / 36 / 0
J2 thumb	19 / 21 / 19	20 / 22 / 20	0 / 2 / 0	19 / 21 / 19	2 / 4 / 2
J3 clip	20 / 28 / 20	26 / 34 / 26	7 / 15 / 7	21 / 29 / 21	15 / 23 / 3
J4 thumb	27 / 28 / 27	17 / 18 / 17	0 / 1 / 0	19 / 20 / 19	5 / 6 / 5
J5 clip	27 / 32 / 27	19 / 24 / 19	0 / 5 / 0	27 / 32 / 27	14 / 19 / 8
Average	**18.6 / 25.8 / 18.6**	**16.4 / 23.6 / 16.4**	**4.6 / 11.8 / 1.4**	**17.2 / 24.4 / 17.2**	**10.4 / 17.6 / 3.6**
Context switches	4	4	5	4	10

SRTF has by far the best averages (waiting 4.6) because it always finishes the shortest remaining work. FCFS, SJF and non-preemptive Priority are poor here because J1, the 20-unit film, arrives first and everything waits behind it for 20 units: the convoy effect the customers complain about.

2.3 Program: Round Robin

rr.c (Appendix A) keeps a circular ready queue, runs the job at the head for min(quantum, remaining), admits arrivals during the slice, and re-queues the job if unfinished. TODO 1 is the body of the main loop. Output for quantum 3 (the Python simulation and the C program agree for every quantum):

```
Round Robin, quantum = 3
Gantt: J1[0-3] J2[3-5] J3[5-8] J4[8-9] J1[9-12] J5[12-15] J3[15-18] J1[18-21] J5[21-23] J3[23-25] J1[25-28] J1[28-31] J1[31-34] J1[34-36]

Job  Arr Burst Done Wait Turn Resp
J1    0   20   36   16   36    0
J2    1    2    5    2    4    2
J3    2    8   25   15   23    3
J4    3    1    9    5    6    5
J5    4    5   23   14   19    8
Average wa ting = 1  .40  tu naround = 17.60  response = 3.60  context switches = 10
```

2.4 Quantum study (1, 3, 5 and 20)

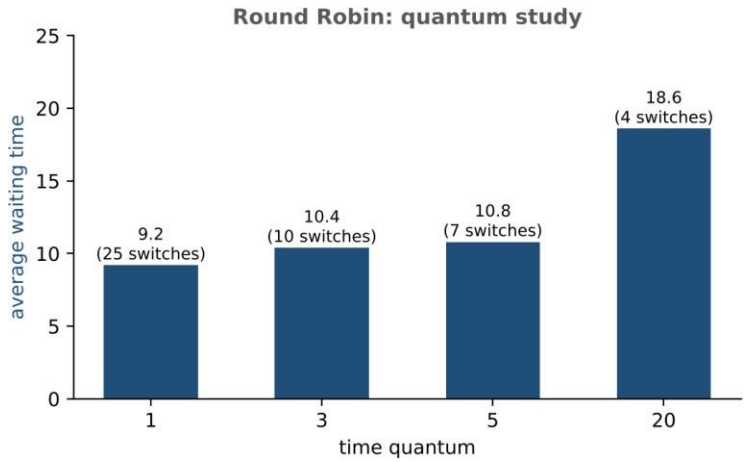


Figure 4. Average waiting time and context switches against quantum

Quantum	Average waiting	Average turnaround	Average response	Context switches
1	9.2	16.4	1.2	25
3	10.4	17.6	3.6	10
5	10.8	18.0	5.4	7
20	18.6	25.8	18.6	4

Program output for the two extremes: quantum 1 gives Average waiting = 9.20 turnaround = 16.40 response = 1.20 context switches = 25; quantum 20 gives Average waiting = 18.60 turnaround = 25.80 response = 18.60 context switches = 4.

A small quantum gives short jobs quick turns (response 1.2) and the lowest waiting, but needs 25 switches, six times as many as quantum 20, and each switch costs real time and cache. A quantum of 20 is longer than

every job but the film, so Round Robin degenerates into FCFS (identical Gantt chart and averages). Quantum 3 or 5 keeps most of the benefit with 10 or 7 switches. The usual rule is a quantum a little longer than a typical short job, so most short jobs finish in one turn.

2.5 Starvation and ageing

SJF, SRTF and Priority can starve a job: if shorter or higher-priority jobs keep arriving, a long or low-priority job never gets the CPU. I simulated it: a low-priority job (priority 9, burst 3) arrives at time 1, then a priority-1 job of burst 2 arrives every 2 time units until time 60.

Policy	Low-priority job starts at
Priority, no ageing	63 (only after all 30 high-priority jobs finished)
Priority with ageing (priority improves by 1 for every 5 units waited)	41

Ageing raises a waiting job's priority over time, so after at most $(9 - 1) \times 5 = 40$ units of waiting it outranks the newcomers and runs. Starvation becomes a bounded delay.

2.6 How a multilevel feedback queue treats short and long jobs

New jobs enter the top queue with a short quantum. A job that finishes within it (a thumbnail) never moves. A job that uses its whole quantum is demoted to the next queue, which has a longer quantum and lower priority. A film therefore sinks to the bottom queue, where it runs in long slices only when nothing shorter is waiting. Short jobs are favoured without anyone having to predict job lengths. My queues use quanta 2, 6 and then FCFS; a new arrival in a higher queue pre-empts a lower one.

Policy	Average wait of thumbnails (J2, J4)	Average wait of long jobs (J1, J3)
FCFS	23.0	10.0
SJF	18.5	13.0
SRTF	0.0	11.5
Priority	19.0	10.5
Round Robin q=3	3.5	15.5
Multilevel feedback queue	2.0	13.5

2.7 Recommendation

Use a multilevel feedback queue with ageing. Its overall average wait is 9.2, the same as Round Robin with quantum 1 but with 8 context switches instead of 25, and its thumbnails wait 2.0 against 23.0 under FCFS. SRTF has better averages (4.6) but needs each job's remaining time, which a video service can only estimate, and it starves long films whenever short jobs keep arriving. The processors stay fully used because the long films fill any gap. Add a periodic boost or ageing so films cannot starve, and measure the real mix of job lengths to set the quanta.

2.8 Alternative analysis: multilevel feedback queue against a single Round Robin queue

Measure	Single Round Robin queue (q=3)	Multilevel feedback queue
Average waiting, short jobs	3.5	2.0
Average waiting, long jobs	15.5	13.5
Context switches	10	8
Scheduling cost per decision	O(1): take the head of one queue	O(1) per queue: look at the head of the highest non-empty queue, k queues (here 3), so O(k) = O(1)
Compare: SJF	O(n) to scan for the shortest job (O(log n) with a heap)	Not needed: queue position replaces the sorting
Starvation risk	None: every job gets a turn each round	Real: long jobs wait while short jobs keep arriving; fixed by ageing or by boosting all jobs to the top queue every few seconds

Q3. Threads (6 marks)

Conclusion. Use a pool of threads sized to the number of cores (plus a few more if frames wait for cameras). On this 2-core machine every pool size gave a speed-up of about 2, and the limit is the number of cores, not the number of threads.

3.1 What threads share and what each has of its own

Shared by all threads of a process	Private to each thread
Code (text) section	Thread ID
Global and static data, and the heap	Program counter and CPU registers
Open files and sockets (descriptor table)	Stack (local variables, return addresses)
Address space and page tables; signal handlers; user ID	Thread-local storage; signal mask; scheduling priority

Sharing is what makes threads cheap and what makes them dangerous: any thread can overwrite another's data.

3.2 User-level and kernel-level threads; the three models

	User-level threads	Kernel-level threads
Managed by	A library in user space; the kernel sees one process	The kernel, which schedules each thread
Create / switch cost	Very low: no system call	Higher: a system call and a kernel switch
A thread blocks in a system call	The whole process blocks	Only that thread blocks; others keep running
Multiple cores	No: the kernel sees one schedulable unit	Yes: threads run on different cores at the same time

Model	Mapping	Strength	Weakness	Example
Many-to-one	Many user threads to one kernel thread	Cheap thread management	One blocking call blocks all; no parallelism	Early Green threads
One-to-one	Each user thread has its own kernel thread	True parallelism; blocking is local	Each thread uses kernel resources, so there is a practical limit on thread count	Linux (pthreads), Windows
Many-to-many	Many user threads multiplexed onto a smaller number of kernel threads	Parallelism without one kernel thread per user thread	Complex to build	Older Solaris; Go's goroutines work this way

POSIX threads on Linux use the one-to-one model, so my programs get real parallelism and each thread appears to the scheduler as a separate task.

3.3 Program and measured speed-up

threads.c (Appendix B) processes 64 camera frames. Each frame stands in for plate reading: a fixed loop calibrated to about 20 ms of CPU time. The program runs it (1) sequentially, (2) with one thread per camera, (3) with pools of 4, 8 and 16 threads that take the next camera from a shared atomic counter, and (4) with pools of 4 and 8 processes. Each thread result is checked against the sequential checksum, so every frame was processed exactly once. I ran the whole experiment three times (the VM is noisy).

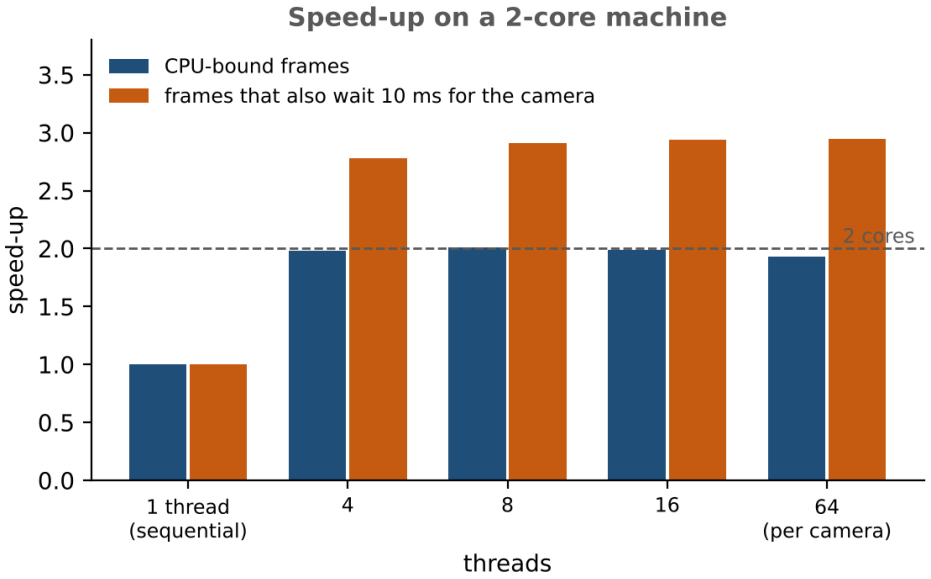


Figure 5. Speed-up over sequential processing on a 2-core machine

Method	Speed-up, CPU-bound frames (3 runs: range / mean)	Speed-up when each frame also waits 10 ms for the camera (2 runs, mean)
Sequential	1.00	1.00
Thread per camera (64 threads)	1.90 to 1.98 / 1.93	2.95
Pool of 4 threads	1.94 to 2.01 / 1.98	2.78
Pool of 8 threads	1.98 to 2.04 / 2.01	2.91
Pool of 16 threads	1.91 to 2.06 / 1.98	2.94
Pool of 4 processes	1.94 to 2.08 / 1.99	2.87
Pool of 8 processes	1.87 to 2.09 / 1.95	2.99

Sequential time was 1.0 to 1.4 s (about 20 ms per frame) and the parallel versions took 0.5 to 0.7 s.

Why the speed-up stops. The machine has 2 cores, so CPU-bound work cannot go faster than 2 times, however many threads there are: Amdahl's bound is the hardware, not the code. Extra threads beyond the core count only add scheduling and cache cost, which is why 16 threads were no faster than 4 (differences are within run-to-run noise). When each frame also waits 10 ms for a camera, a waiting thread frees its core for another one, so the speed-up rises to about 3. The limit then is the ideal 64 × 20 ms ÷ 2 cores = 0.64 s, against the sequential 64 × 30 ms = 1.9 s, a ratio of 3.0, matching the measured 2.8 to 3.0. On a machine with 8 cores the same program would scale to about 8, and the design question would become memory bandwidth and the shared plate database.

3.4 Unprotected shared counters

Two threads each added 1 to a shared counter 5,000,000 times (expected 10,000,000):

Protection	Run 1	Run 2	Run 3
None (plain counter++)	9,771,224	9,829,216	9,769,980
Mutex	10,000,000	10,000,000	10,000,000
Atomic fetch-and-add	10,000,000	10,000,000	10,000,000

Without protection about 2% of updates were lost. counter++ is three steps (load, add, store). If both threads load the same value, both store value + 1, and one update disappears. Which updates are lost changes from run to run, which is why such bugs are hard to reproduce. A mutex makes the three steps indivisible; an atomic instruction does it in hardware.

3.5 Alternative: several processes instead of threads

Measure	Threads	Processes
Creation time	52 µs each (measured on this machine earlier)	183 µs each with a small parent; 7.3 ms with a 256 MB parent (fork copies page tables)
Switching time	1.6 to 2.5 µs per switch	1.9 to 2.4 µs per switch (measured in Q1); no significant difference on one CPU
Memory	One address space; each thread adds a stack	Separate address spaces: page tables and, once pages are written, private copies
Combining results	Shared variables with a lock or atomic add	IPC: each process sends its total through a pipe; the parent adds them
Measured speed-up, 4 workers	1.98 (pool of 4)	1.99 (pool of 4)

For coarse work (20 ms frames) the creation cost of a process (0.2 ms) is only 1% of a frame, so both finish equally fast. Threads are better when workers must share large read-only data such as the plate-pattern database or a result list, because nothing is copied. Processes are better when a crash in one frame decoder must not take down the others. The measured processes combined their results through a pipe each, which needed extra code but no locks.

Q4. Inter-Process Communication (6 marks)

Conclusion. For a weather station that exchanges small readings, a pipe is the right method: it needs no extra synchronisation and tells the reader when the data ends. Shared memory with semaphores is about four times faster for 100 MB but only matters at rates above 1 GB/s.

4.1 Pipe implementation

pipe_demo.c (Appendix A): the parent is the sensor process and the child is the analyser. The parent writes 1,000 readings (a 16-byte structure with an id and a temperature) into the pipe; the child reads them until read() returns 0. TODO 1 is that read loop. Output:

```
analyser: got 1000 readings, mean temperature 24.50
sensor: sent 1000 readings, analyser finished
```

One mistake I made and fixed: the child ends with _exit(0), which does not flush stdio, so with output redirected to a file its message was lost until I added fflush(stdout). This is the same buffering effect as in the shell experiment in Assignment 1.

4.2 How the pipe signals end of data

read() on a pipe blocks while the pipe is empty but a write end is still open. It returns 0 (end of file) only when every write end of the pipe has been closed. I confirmed this with strace on the normal run; the last calls in the analyser are:

```
read(3, "", 16)                 = 0
close(3)                        = 0
```

Removing close(fd[1]) in the parent

I compiled the program with that line removed and ran it under a 3-second timeout. It did not finish: the exit code was 124, meaning the timeout killed it. The reason is a circular wait. The parent still holds the write end open while it waits in waitpid() for the child. The child's read() only returns 0 once all write ends are closed, so it waits for data that never comes, and the parent waits for the child. Neither can proceed. The child must also close its own copy of the write end for the same reason, which the program does.

4.3 Shared memory against message passing

	Shared memory	Message passing (pipe, message queue)
Model	Both processes map the same physical pages and read and write them directly	Processes send and receive messages through the kernel
Data copying	None after setup: the producer writes into the region and the consumer reads it in place	Two copies: from the sender's buffer into the kernel, then into the receiver's buffer
Synchronisation	Must be added by the programmer: semaphores or mutexes decide when a slot is full or empty	Built in: the kernel blocks a reader when empty and a writer when full, and keeps messages in order
Kernel involvement	Only for setup (and semaphore waits under contention)	On every send and receive (a system call each)
Best for	Large, frequent transfers between processes on one machine	Small messages; also works across machines

Shared memory needs extra synchronisation because the kernel does not know when the data is ready: without a rule, the consumer could read a slot while the producer is half-way through writing it, or the producer could overwrite a slot not yet read. Pipes and queues avoid this because the kernel serialises access.

4.4 Growth investigation: 1 MB, 10 MB and 100 MB

ipc_bench.c (Appendix B) sends the data from one process to another with a pipe (64 KiB writes), a POSIX message queue (8 KiB messages, the system limit) and shared memory (a ring of 16 slots of 64 KiB guarded by two semaphores). The producer generates each block and the consumer sums it, so both do real work, and the sum is checked at the end: no run reported a data error. Best of three runs:

Data size	Pipe (s)	Message queue (s)	Shared memory (s)	Throughput MB/s: pipe / queue / shared
1 MB	0.0007	0.0006	0.0008	1,439 / 1,738 / 1,246
10 MB	0.0039	0.0045	0.0018	2,585 / 2,241 / 5,524
100 MB	0.0381	0.0362	0.0099	2,622 / 2,761 / 10,105

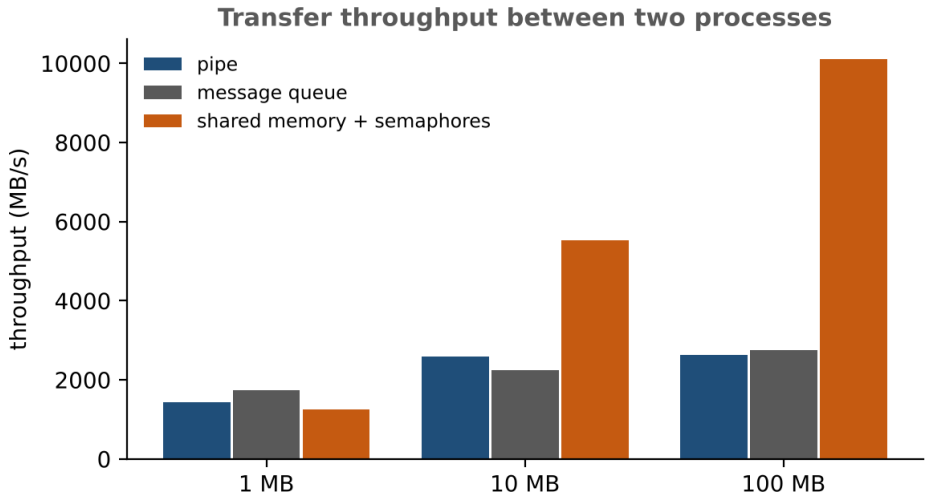


Figure 6. Transfer throughput by method and size

Evaluation. Time grows linearly with the size for the kernel-copying methods (100 MB took 9.8 times as long as 10 MB for the pipe, close to the 10 times expected). At 1 MB the fixed cost of forking the process and starting the transfer dominates, so all three look the same. At 100 MB shared memory is 3.9 times faster than the pipe, because it avoids the two kernel copies. The message queue performed about the same as the pipe in this test: both pay a system call and two copies for every block. The shared-memory advantage is partly hidden because both methods still do the same generating and summing work.

Choice. A weather station produces readings of a few bytes at a few per second, far below 1 MB/s, so any method keeps up and the simplest wins: a pipe, which also gives the clean end-of-data signal and has no data-loss risk (a full pipe makes the sensor wait rather than drop readings). Shared memory would be chosen if the analyser needed continuous high-rate data such as images from many sensors.

4.5 Alternative: shared memory with a semaphore for each reading

Measure	Pipe	Shared memory and semaphore
Copies per reading	2 (user to kernel, kernel to user)	0 (the analyser reads the slot in place)
Time to transfer n bytes	About 2n / copy bandwidth plus one system call pair per block: measured 38 ms for 100 MB	About n / memory bandwidth plus semaphore operations: measured 9.9 ms for 100 MB with 64 KiB slots
Synchronisation	None to write	Two semaphores (empty, full) per ring; a missing sem_post stops the pipeline, a missing sem_wait corrupts data
Cost when readings are tiny	One system call per reading, unless batched	Semaphore post and wait per reading (a system call only when the other side is asleep); batching many readings per slot removes most of it

My benchmark synchronised once per 64 KiB, not per reading. Per 16-byte reading the semaphore handshake would cost far more than the copy it saves, so for readings the right design is to batch many into each slot, or just use the pipe.

Q5. Synchronisation: Critical Sections, Semaphores and Monitors (11 marks)

Conclusion. Starting from the three requirements for a critical section, I derive a design that protects each coach with its own lock, acquired in a fixed order. It sold no seat twice and had 3.1 times the throughput of one global lock at 4 threads.

5.1 From the principle to a design

The principle says a solution must guarantee three things. Applied to the booking system, where the critical section is "check that a seat is free, then mark it sold":

Requirement	Meaning	In the booking system
Mutual exclusion	At most one process is inside the critical section at a time	Two agents must never both see seat 14C as free and both sell it
Progress	If no one is inside, only processes that want to enter decide who goes next, and the decision cannot be postponed forever	If no one is booking coach 3, an agent who wants it is let in at once; agents booking other coaches do not influence that decision
Bounded waiting	There is a limit on how many times others may enter before a waiting process gets in	No agent can be overtaken forever; semaphores wake waiters in first-in order

The design steps that follow: (1) identify the shared data: seat table per coach; (2) make the check-and-mark a critical section; (3) choose the mutual-exclusion tool (semaphore or mutex); (4) choose how big the protected region is (the deployment decision in section 5.7), because the larger it is, the less parallelism remains.

5.2 Peterson's solution

```
shared: flag[2] = {false, false}, turn
process i (j = the other process):
    flag[i] = true;           /* I want to enter */
    turn = j;                 /* but you go first if you want to */
    while (flag[j] && turn == j) ;     /* wait */
    ... critical section ...
    flag[i] = false;
```

- **Mutual exclusion.** Suppose both processes are inside. Then both flags are true. Each passed its test only because turn pointed at itself. But turn holds one value, and the process that wrote it last pointed it at the other process, so that process could not have passed while the other's flag was set. This is a contradiction.
- **Progress.** If only one process wants to enter, the other's flag is false and the loop falls through immediately. If both want to enter, turn decides, so the choice does not depend on a process outside the competition.
- **Bounded waiting.** After process i leaves and tries again, it sets turn = j, handing priority to a waiting j. So j waits for at most one entry by i.

Why it may fail on modern processors

The proof assumes that each process sees the other's writes in program order. Modern processors break this. A store goes into a store buffer and a later load may complete before it; the compiler may also reorder. Then

process i's flag[i] = true is not yet visible when it reads flag[j], so both read false and both enter. I tested it: two threads each entered the critical section 5,000,000 times, updating a shared counter.

Version	Run 1: lost updates	Run 2	Run 3
Plain volatile variables	4,190	13,725	11,811
Sequentially consistent atomics (memory fences)	0	0	0

Every lost update is a moment when both threads were inside the critical section. The fix is a memory barrier (or sequentially consistent atomics) between the writes and the reads, which is what the second row uses. In practice one uses the operating system's locks, which include the barriers.

5.3 Semaphores protect the seat counter

A semaphore S is an integer with two atomic operations: wait(S) (decrement; sleep if the result would be negative) and signal(S) (increment; wake a sleeper).

```
binary semaphore mutex = 1;        /* mutual exclusion */
wait(mutex);
if (seats_left > 0) { seats_left--; sell_ticket(); }
signal(mutex);

counting semaphore seats = N;      /* counts free seats, no separate counter */
if (try_wait(seats)) sell_ticket();   /* sold out at 0 */
```

The binary semaphore (initial value 1) gives mutual exclusion over the check-and-decrement. A counting semaphore (initial value = number of seats) lets the semaphore itself be the seat count: each successful wait takes a seat, and the call fails once it reaches zero, so no explicit test is needed.

5.4 Classic problems with semaphores

Bounded buffer (producer–consumer)

```
semaphore empty = SIZE, full = 0, mutex = 1;
producer:  wait(empty); wait(mutex);
           buf[in] = item; in = (in+1) % SIZE;
           signal(mutex); signal(full);
consumer:  wait(full);  wait(mutex);
           item = buf[out]; out = (out+1) % SIZE;
           signal(mutex); signal(empty);
```

empty counts free slots and blocks a producer when the buffer is full; full counts items and blocks a consumer when it is empty; mutex protects the indices. The order matters: waiting on the mutex before the counting semaphore could deadlock (a producer holding the mutex while waiting for space).

Readers–writers (readers have priority)

```
semaphore rw = 1, mutex = 1;  int readcount = 0;
reader:  wait(mutex); if (++readcount == 1) wait(rw); signal(mutex);
         ... read ...
         wait(mutex); if (--readcount == 0) signal(rw); signal(mutex);
writer:  wait(rw);  ... write ...  signal(rw);
```

Many readers (agents looking at seat availability) may read together; the first reader locks writers out and the last one lets them in. A writer (a booking) has exclusive access. This version can starve writers if readers keep arriving, so a booking system would use a fair variant.

Dining philosophers

Five philosophers share five forks; each needs two. If all pick up their left fork at once, they wait forever for the right one: deadlock. A standard fix is a semaphore room initialised to 4 so at most four sit down, so at least one always gets both forks. I ran both versions (dining.c, Appendix B), with each philosopher eating 2,000 times:

Version	Result
With room semaphore (max 4 seated)	All five finished; 10,000 meals eaten (expected 10,000)
Naive (pick left, pause 100 µs, pick right)	Deadlocked: killed by the 5-second timeout (exit code 124)

5.5 A monitor with condition variables

A monitor bundles the shared data with the only functions allowed to touch it, and the language or library guarantees that one thread at a time is inside. Condition variables let a thread wait inside the monitor for a condition and release the monitor while waiting.

```
monitor BoundedBuffer {
   int count = 0;  condition not_full, not_empty;
   void put(item x) {
      while (count == SIZE) wait(not_full);
      insert(x); count++; signal(not_empty);
   }
   item get() {
      while (count == 0) wait(not_empty);
      x = remove(); count--; signal(not_full); return x;
   }
}
```

It is easier to get right because mutual exclusion is automatic (no forgotten signal(mutex)), the waiting condition is written as a plain test on the data, and there is no ordering trap between two semaphores. The while (not if) re-checks the condition after waking up. In C the same pattern is pthread_mutex_t plus pthread_cond_t.

5.6 Program and test evidence: bounded buffer with semaphores

bounded.c (Appendix A) has a buffer of 5 slots, P producers and C consumers. Together the producers generate the items 1 to 20 (sum 210). TODO 1 is the producer's sem_wait(&empty) / insert / sem_post(&full) and TODO 2 is the consumer's mirror image, with a mutex around the buffer indices. The program counts how many times each item is consumed and checks that every item appears exactly once.

```
$ ./bounded 1 1
producers=1 consumers=1: sum=210 every item consumed exactly once: yes (expected 210)
$ ./bounded 4 4
producers=4 consumers=4: sum=210 every item consumed exactly once: yes (expected 210)
```

Configuration	Runs with sum 210 and every item exactly once
1 producer, 1 consumer, with mutex	100 of 100
4 producers, 4 consumers, with mutex	100 of 100

Removing the mutex

With only 20 items the race is rare. Without the mutex, 1 producer and 1 consumer was still correct (100 of 100) because they never touch the same index, 4 and 4 was correct in 99 of 100 runs, and 8 and 8 in 100 of 100. That means the test is too short to prove anything, not that the code is safe. I therefore repeated it with 200,000 items (expected sum 20,000,100,000), using 4 producers and 4 consumers:

Version	Run 1	Run 2	Run 3	Every item once?
With mutex	20,000,100,000	20,000,100,000	20,000,100,000	Yes, all runs
No mutex	20,000,300,512	20,000,308,692	20,000,378,629	No, all runs

Without the mutex the result is wrong every time. The semaphores still keep the counts of full and empty slots right, but two producers can read the same in index, write to the same slot (one item is lost) and both advance in once, while two consumers can read the same item (it is counted twice). The sum is too large because of the duplicated reads.

5.7 Deployment decision: one lock, one lock per train, or one lock per coach

booking.c (Appendix B) models 20 trains of 10 coaches of 72 seats. Each request picks a random seat and, while holding its lock, checks the seat, does 2,000 iterations of simulated fare and payment work (about a microsecond), and marks the seat sold. Every configuration was run 7 times; the table gives the median throughput in millions of bookings per second. The last column counts double-sold seats (tickets sold minus seats actually taken) summed over the 7 runs.

Locking	1 thread	2 threads	4 threads	8 threads	Double sales (2, 4, 8 threads)
No lock	2.50	3.94	3.44	2.70	9, 14, 17
One global lock	1.47	0.82	0.96	1.31	0, 0, 0
One lock per train (20 locks)	1.42	1.58	1.69	1.86	0, 0, 0
One lock per coach (200 locks)	1.49	**2.54**	**2.97**	2.44	0, 0, 0

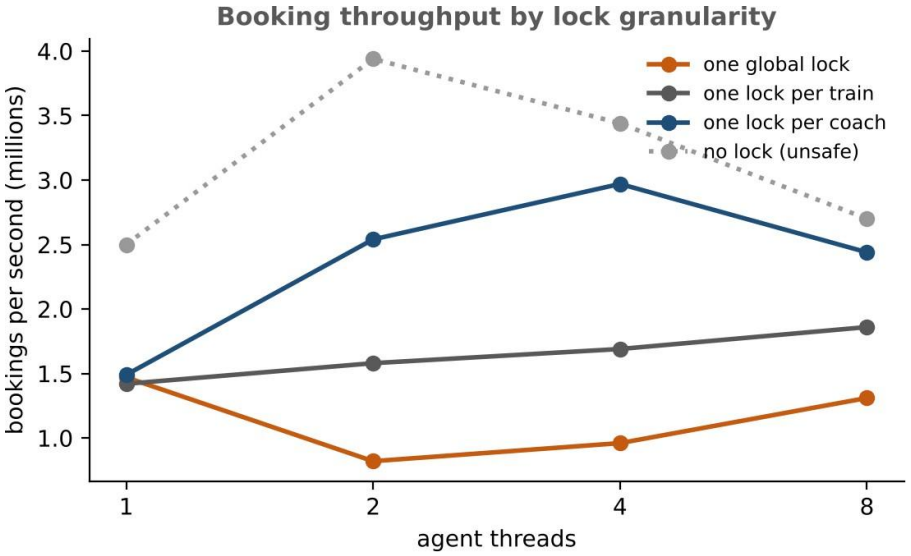


Figure 7. Median booking throughput by lock granularity (millions per second)

Reading the results. With no lock the same seat was sold twice, which is the original fault. A global lock forces every booking through one queue: with 2 threads throughput even fell below one thread (0.82 against 1.47) because of lock handoff and waiting. The lock per train improved slowly (1.86 at 8 threads). The lock per coach was fastest from 2 threads on, reaching 2.97 at 4 threads, 3.1 times the global lock. It does not exceed the no-lock rate, which is the upper limit here, and the machine has only 2 cores, so gains above about 2 times a single thread cannot appear at 8 threads.

Decision: one lock per coach. It removes the double sales, keeps 72 seats' worth of work inside a short critical section, and the 200 mutexes take only about 8 KB. Conditions for using it safely: (1) a booking that touches several coaches (a family on two coaches, a return journey) must take its locks in a fixed order (train number, then coach number) so two such bookings can never wait for each other in a cycle; (2) the lock must cover the check-and-mark as one step, never just the mark; (3) a train-level total such as "seats left" should be an atomic counter or derived, not guarded by a global lock. The per-train lock is the fallback if multi-coach bookings dominate, since it needs no ordering rule. The measurements are from a 2-core virtual machine with simulated work, so the ratio should be re-measured on the real server.

5.8 Alternative: atomic operations or a spin lock instead of a semaphore and mutex

counter.c (Appendix B) updates one shared counter with an atomic fetch-and-add, a spin lock (atomic test-and-set in a loop), a mutex and a binary semaphore. Time per update (ns), with 4,000,000 updates in total, an empty critical section, and CPU time in seconds in brackets:

Method	1 thread	4 threads	16 threads
Atomic fetch-and-add	8.1 (0.03 s)	15.0 (0.11 s)	17.3 (0.13 s)
Spin lock	10.3 (0.04 s)	67.2 (0.50 s)	280.8 (2.23 s)
Mutex	27.9 (0.11 s)	57.3 (0.40 s)	69.5 (0.51 s)
Semaphore	26.7 (0.11 s)	231.1 (1.72 s)	295.5 (2.29 s)

For this tiny critical section the atomic instruction is fastest at every thread count and wastes almost no CPU time. The spin lock is quick when alone (10 ns) but at 16 threads on 2 cores it takes 281 ns per update and used 2.2 CPU-seconds for 0.28 s of wall time, because spinning threads burn a whole core while the thread

holding the lock is waiting to be scheduled. The mutex scales better under contention because waiters sleep. The semaphore is the slowest under contention in this test.

Now a long critical section of 20 µs (20,000 updates in total, so the useful work is 0.4 s):

Method	1 thread wall / CPU (s)	4 threads	16 threads
Spin lock	0.43 / 0.42	0.76 / 1.48	2.33 / 4.62
Mutex	0.42 / 0.42	0.60 / 0.64	0.56 / 0.59
Semaphore	0.44 / 0.42	0.69 / 0.76	0.70 / 0.79

With 16 threads the spin lock burned 4.6 CPU-seconds for 0.4 seconds of useful work, about 4.2 seconds wasted spinning, and it was also the slowest in wall-clock time. The mutex used 0.59 CPU-seconds, almost no waste. (The atomic method has no row because an atomic add cannot guard a multi-step critical section.)

Method	Suits	Does not suit
Atomic operations	A single counter or flag, such as a seat count (a compare-and-swap loop can implement “decrement only if greater than zero”)	Anything with several steps or several variables, such as check seat, then charge payment, then mark
Spin lock	Critical sections shorter than a context switch (a few instructions), when threads never outnumber cores	Long critical sections, many threads, or anywhere a lock holder can sleep or be pre-empted
Mutex / semaphore	Long or variable critical sections: waiters sleep and use no CPU	The shortest critical sections, where the system call cost dominates (27.9 ns against 8.1 ns here)

For the booking system, whose critical section includes fare and payment checks, a mutex is right and a spin lock would be wrong.

Q6. Deadlock Handling and the Banker's Algorithm (13 marks)

Conclusion. The cluster should avoid deadlock with the Banker's algorithm at job admission (jobs already declare their maximum demand when submitted), add resource ordering where jobs take locks themselves, and keep periodic detection as a backup. The Banker's check costs at most 2.2 ms at 1,000 jobs, which is negligible.

A. Characterise deadlock (2 marks)

Four conditions must hold at the same time for deadlock. Here they are in the cluster, with the three jobs of the graph below:

Condition	Meaning	In the research cluster
Mutual exclusion	A resource can be used by only one job at a time	A GPU, a simulation licence and a fast-storage slot are each used by one job at a time
Hold and wait	A job holds some resources while waiting for others	J1 holds a GPU and waits for a licence; J2 holds the licence and waits for storage
No pre-emption	A resource cannot be forcibly taken from the job holding it	A running simulation cannot lose its GPU or storage without losing its work
Circular wait	A closed chain of jobs, each waiting for a resource held by the next	J1 → licence (held by J2) → storage (held by J3) → GPU (held by J1) → J1

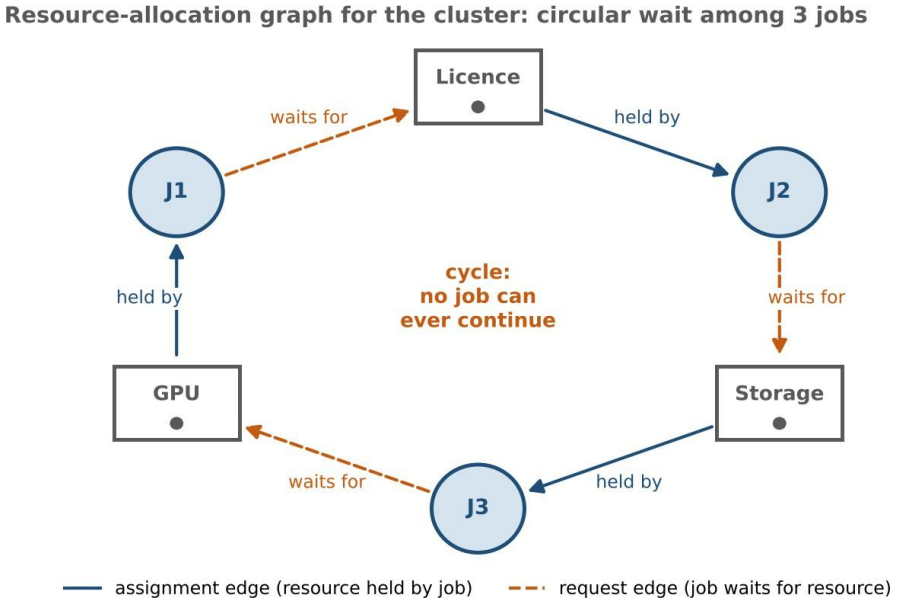


Figure 8. Resource-allocation graph with a circular wait among three jobs

Each resource has a single instance, so a cycle in the graph means a deadlock: no job in the cycle can ever continue. Removing any one of the four conditions makes deadlock impossible, and that is the idea behind prevention.

B. Compare the strategies (3 marks)

Strategy	How it works	Fit for the research cluster	Verdict
Prevention	Make one condition impossible. Mutual exclusion: cannot remove for GPUs and licences. Hold and wait: request everything at once. No pre-emption: take resources back. Circular wait: number the resource types and require requests in increasing order	Request-all-at-once is simple but leaves GPUs idle while a job waits for storage it needs only later. Pre-emption works for storage (checkpoint) but not for a running licence-bound simulation. Ordering (GPU, then licence, then storage) is cheap and removes cycles	Good for ad-hoc locks: use resource ordering. Request-all-at-once too wasteful
Avoidance (Banker's)	Grant a request only if the resulting state is safe; needs each job's maximum demand in advance	Jobs already state GPU, licence and storage needs when submitted. Costs $O(m \times n^2)$ per request, cheap at this scale. Needs honest maxima and a fixed set of resources	Best fit at the scheduler
Detection and recovery	Allow deadlock, run a detection algorithm periodically, then abort or roll back jobs	No advance knowledge needed and no per-request cost. Idle holding until detected; aborting loses work unless jobs checkpoint	Good as a backup
Ignore (ostrich)	Assume deadlock is rare; reboot or intervene	The stated problem is that the whole cluster stops until an administrator intervenes, so deadlock is not rare here and its cost is high	Rejected

C. Apply the Banker's algorithm (4 marks)

This part also answers the "Complete the Code" and "show by hand" questions under A, because they use the same program. The resources are A = GPUs (10 instances), B = simulation licences (5) and C = fast-storage slots (7). The state is the standard textbook state, since the sheet did not supply one:

Process	Allocation (A B C)	Max	Need = Max − Allocation
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1
Available	**3 3 2**		

banker.c (Appendix A) implements the **safety algorithm** and the **resource-request algorithm**. Safety: set Work = Available; repeatedly find an unfinished process whose Need is at most Work, pretend it finishes and add its Allocation to Work; the state is safe if every process can finish in some order. Request: refuse if the request exceeds the process's Need (error) or the Available vector (the process waits); otherwise pretend

to grant it, run the safety algorithm, and keep the grant only if the new state is safe. TODO 1 is the body of the safety algorithm. Output:

```
 Safety algorithm: SAFE, sequence <P1, P3, P4, P0, P2>

Request 1: P1 asks (1,0,2)
 granted: new state is safe, e.g. <P1, P3, P4, P0, P2>
Request 2: P4 asks (3,3,0)
 refused: not enough available now, P4 must wait
Request 3: P0 asks (0,2,0)  [state now includes P1's grant]
 refused: granting would leave an UNSAFE state, P0 must wait

For comparison, the same P0 request (0,2,0) in the ORIGINAL state:
 granted: new state is safe, e.g. <P3, P1, P2, P0, P4>
```

- **Safe sequence.** <P1, P3, P4, P0, P2> is valid; the textbook's <P1, P3, P4, P2, P0> is also valid. A safe state usually has more than one safe sequence, and the program reports the first it finds. Hand check: Work = (3,3,2); P1 (need 1 2 2) runs, Work = (5,3,2); P3 (0 1 1) runs, Work = (7,4,3); P4 (4 3 1) runs, Work = (7,4,5); P0 (7 4 3) runs, Work = (7,5,5); P2 (6 0 0) runs, Work = (10,5,7), the total.
- **P1 asks (1,0,2): granted.** The request is within P1's need (1 2 2) and Available (3 3 2); the resulting state still has a safe sequence.
- **P4 asks (3,3,0): refused.** It asks for 3 GPUs but only 2 are available after P1's grant (Available = 2 3 0), so P4 must wait.
- **P0 asks (0,2,0): refused, unsafe** (see the hand analysis below).

Why P0's request for (0, 2, 0) leads to an unsafe state (by hand)

Take the state after P1's grant: Available = (2,3,0); P1 now has Allocation (3,0,2) and Need (0,2,0). Check the request: (0,2,0) ≤ Need of P0 (7,4,3) and ≤ Available (2,3,0), so it is legal and we pretend to grant it:

Process	Allocation	Need
P0	0 3 0	7 2 3
P1	3 0 2	0 2 0
P2	3 0 2	6 0 0
P3	2 1 1	0 1 1
P4	0 0 2	4 3 1
Available	**2 1 0**	

Safety test with Work = (2,1,0): P0 needs (7,2,3), and 7 is more than 2; P1 needs (0,2,0), and 2 is more than the 1 licence free; P2 needs (6,0,0), and 6 is more than 2; P3 needs (0,1,1), and the 1 storage slot it needs is not free (C = 0); P4 needs (4,3,1) and 4 is more than 2. No process can finish, so no safe sequence exists. The state is unsafe: it is not a deadlock yet, but if the processes then asked for their maximum demands the cluster could deadlock, so the Banker refuses the request and P0 waits. Granting the same request in the original state (Available 3 3 2) is safe because P3 can still finish first, as the last line of the output shows.

Alternative analysis (set under A): detection and recovery instead of the Banker's algorithm

If deadlocks are allowed to happen and a detection algorithm runs every few minutes, an unsafe request would simply be granted. The detection algorithm uses the same structure as the safety algorithm but uses the jobs' current requests instead of their maximum needs, so it also costs $O(m \times n^2)$.

Measure	Banker's algorithm (avoidance)	Detection and recovery
Time per request	A safety check on every request: $O(m \times n^2)$; measured 7.9 µs typical and 2.2 ms worst case at $n = 1{,}000$, $m = 3$	None: requests are granted at once
Cost of detection	None extra	One $O(m \times n^2)$ run every few minutes (about 2 ms at $n = 1{,}000$): negligible
If 100 requests arrive per second at $n = 1{,}000$	100×2.2 ms = 0.22 CPU-seconds per second (22% of one core) in the worst case; 0.08% typical	A few milliseconds per run, a few times an hour
Cluster behaviour	Never deadlocks; some requests wait even though they would have been safe	Deadlocked jobs sit holding GPUs and licences, idle, until the next detection run (up to the interval) and then recovery
Work lost	None	The aborted job's work since its last checkpoint. Example (assumed): a simulation 4 hours in on 4 GPUs without checkpoints loses 16 GPU-hours; with hourly checkpoints, at most 4 GPU-hours

D. Predict before implementing (1 mark)

Prediction. The safety algorithm sweeps over the n processes, comparing m resources for each; in the worst case it must repeat the sweep n times, because only one more process becomes able to finish per sweep. So the worst-case time is proportional to $m \times n^2$: 10 times more processes should take about 100 times longer, and doubling m should about double the time. In a typical state, where most processes can finish in the first sweep, one or two sweeps suffice and the time should grow only in proportion to $m \times n$.

Test. The worst-case state has a chain: only the last process can finish first, then the one before it, and so on, with the binding resource last in the vector so all m comparisons are made. The typical state lets everyone finish in one sweep.

E. Measure (1 mark)

The program times the safety algorithm many times per case and reports the average per check ($m = 3$ unless stated):

State	n = 10	n = 100	n = 1,000	Growth per 10× n
Worst case (chain)	0.34 µs	23.38 µs	2,222.74 µs	×69, then ×95 (n^2 predicts ×100)
Typical	0.10 µs	0.82 µs	7.85 µs	×8.2, then ×9.6 (n predicts ×10)

Worst case at n = 1,000	m = 3	m = 6	m = 12
Time per check	2,429 µs	4,573 µs	6,839 µs

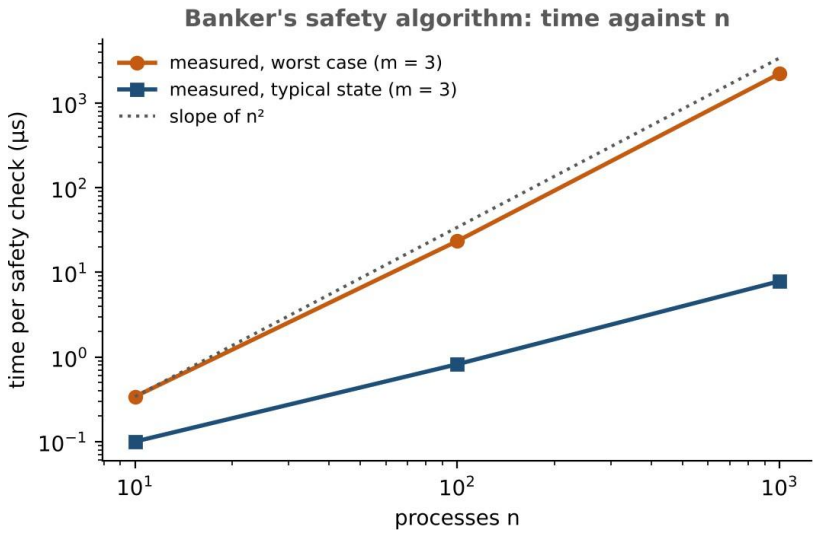


Figure 9. Banker's safety check: time against number of processes (both axes logarithmic)

Comparison with the prediction. The slope on the logarithmic chart is 1.84 from 10 to 100 and 1.98 from 100 to 1,000, close to the predicted 2 (the first interval is lower because fixed costs matter at n = 10). The typical case has a slope of 0.91 and 0.98, close to 1. So the worst case is quadratic and the typical case is linear in n. The dependence on m is positive but weaker than proportional: going from m = 3 to m = 12 (four times) made the check 2.8 times slower, because the loop overhead and the sweep itself do not depend on m. The prediction (worst case $m \times n^2$) is a correct upper bound, and its n^2 growth is confirmed.

F. Defend the final design (2 marks)

Recommendation: use avoidance as the main strategy, with two supporting measures.

1. **Banker's algorithm in the job scheduler.** Every job already declares its maximum GPUs, licences and storage on submission, and only the scheduler hands out those resources. A request that leaves an unsafe state waits in the queue. Cost: at most 2.2 ms per check at 1,000 jobs in the worst case and 7.9 µs in a typical state (measured), which is trivial next to jobs that run for minutes or hours. Even at 10,000 jobs, the worst-case check (100 times more, by the n^2 law) is about 220 ms, still small against a scheduling cycle of seconds, and one could split the cluster into partitions if it grows beyond that.
2. **Resource ordering for locks that jobs take themselves** (for example two jobs that both lock a licence and a dataset): GPU before licence before storage. It costs nothing at run time and removes circular wait.
3. **Periodic detection every few minutes** as a safety net, with job checkpointing so an abort costs at most the work since the last checkpoint.

Critical evaluation of the cost and the assumptions. The Banker's algorithm is only as good as its inputs:

- **Maximum demands must be known in advance.** Researchers over-estimate (safe but wasteful: resources reserved in the safety test are not available to others, so utilisation drops) or under-estimate (the assumption fails, and a job may ask for more than it declared; the scheduler must kill or refuse it). Mitigation: take the maximum from the submission script, enforce it at run time, and review typical over-declaration.

- **It assumes a fixed number of resources and jobs that give resources back in finite time.** If a GPU fails, or a licence server goes down, the safety calculation is no longer true. Detection is the backup for these cases.
- **It is conservative.** It refuses some requests that would not have deadlocked, so jobs wait longer than strictly necessary. For the cluster, waiting is cheaper than a whole-cluster stop.
- **Cost grows as n squared.** Measured growth confirms it: fine at 1,000 jobs, but it must be watched as the cluster grows.

Ignoring the problem was rejected because the whole cluster stops. Detection alone was rejected because jobs would sit holding expensive GPUs until the next run and then lose work. Prevention alone (request everything at once) would waste GPUs. A combination of avoidance at admission, ordering for ad-hoc locks and detection as a backup gives the lowest expected loss of work and of GPU time.

Appendix A. Source Code for the Required Programs

Compile with gcc -O2 -Wall (add -pthread for the threaded programs and -lrt for message queues). Files are shown as run.

Q1: forkdemo.c

```
/* forkdemo.c - Q1: create N children with fork(); each sleeps so ps and /proc can observe it */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char *argv[]) {
    int n = argc > 1 ? atoi(argv[1]) : 3;
    printf("parent pid=%d\n", getpid()); fflush(stdout);
    for (int i = 0; i < n; i++) {
        pid_t p = fork();
        if (p < 0) { perror("fork"); exit(1); }
        if (p == 0) {                       /* child: a "student session" */
            printf("child %d: pid=%d ppid=%d\n", i, getpid(), getppid()); fflush(stdout);
            sleep(4);                       /* waiting state (blocked in nanosleep) */
            _exit(10 + i);                  /* exit status 10,11,12 */
        }
    }
    int st; pid_t w;
    while ((w = wait(&st)) > 0)             /* parent reaps all children */
        printf("parent: child %d finished, status %d\n", w, WEXITSTATUS(st));
    return 0;
}
```

Q2: rr.c (Round Robin)

```
/* rr.c - Round Robin scheduler simulation. usage: ./rr quantum */
#include <stdio.h>
#include <stdlib.h>
#define N 5
struct job { const char *name; int arrival, burst, remaining, completion, first; };
int main(int argc, char *argv[]) {
    int q = argc > 1 ? atoi(argv[1]) : 3;
    struct job j[N] = {{"J1",0,20},{"J2",1,2},{"J3",2,8},{"J4",3,1},{"J5",4,5}};
    int queue[1000], head = 0, tail = 0, t = 0, done = 0, next = 0, last = -1, switches = 0;
    for (int i = 0; i < N; i++) { j[i].remaining = j[i].burst; j[i].first = -1; }
    printf("Round Robin, quantum = %d\nGantt: ", q);
    while (next < N && j[next].arrival <= t) queue[tail++] = next++;
    while (done < N) {
        if (head == tail) { t++; while (next < N && j[next].arrival <= t) queue[tail++] = next++; continue; }
        int i = queue[head++];
        /* TODO 1 (completed): run job i for min(q, remaining), admit arrivals, requeue if unfinished */
        int run = j[i].remaining < q ? j[i].remaining : q;
        if (j[i].first < 0) j[i].first = t;
        if (last != -1 && last != i) switches++;
```

```
        last = i;
        printf("%s[%d-%d] ", j[i].name, t, t + run);
        for (int k = 0; k < run; k++) {          /* arrivals during the slice enter the queue first */
            t++; j[i].remaining--;
            while (next < N && j[next].arrival <= t) queue[tail++] = next++;
        }
        if (j[i].remaining > 0) queue[tail++] = i;  /* back of the queue */
        else { j[i].completion = t; done++; }
    }
    double sw = 0, st = 0, sr = 0;
    printf("\n\nJob  Arr Burst Done Wait Turn Resp\n");
    for (int i = 0; i < N; i++) {
        int turn = j[i].completion - j[i].arrival, wait = turn - j[i].burst, resp = j[i].first - j[i].arrival;
        printf("%-4s %3d %5d %4d %4d %4d %4d\n", j[i].name, j[i].arrival, j[i].burst, j[i].completion, wait,
turn, resp);
        sw += wait; st += turn; sr += resp;
    }
    printf("Average waiting = %.2f  turnaround = %.2f  response = %.2f  context switches = %d\n", sw/N,
st/N, sr/N, switches);
    return 0;
}
```

Q4: pipe_demo.c

```
/* pipe_demo.c - Q4: sensor process (parent) sends readings through a pipe; analyser (child) reads until
EOF */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
struct reading { int id; double temp; };
int main(int argc, char *argv[]) {
    int n = argc > 1 ? atoi(argv[1]) : 1000, fd[2];
    if (pipe(fd) < 0) { perror("pipe"); return 1; }
    pid_t p = fork();
    if (p == 0) {                           /* analyser */
        close(fd[1]);                       /* child must close its copy of the write end */
        struct reading r; long count = 0; double sum = 0;
        /* TODO 1 (completed): read until read() returns 0 = end of data */
        while (read(fd[0], &r, sizeof r) == sizeof r) { count++; sum += r.temp; }
        printf("analyser: got %ld readings, mean temperature %.2f\n", count, count ? sum / count : 0);
        close(fd[0]); fflush(stdout); _exit(0);  /* _exit does not flush stdio */
    }
    close(fd[0]);                           /* sensor only writes */
    for (int i = 0; i < n; i++) { struct reading r = { i, 20.0 + (i % 10) }; write(fd[1], &r, sizeof r); }
#ifndef NO_CLOSE
    close(fd[1]);                           /* signals end of data: the last write end is closed */
#endif
    waitpid(p, NULL, 0);
    printf("sensor: sent %d readings, analyser finished\n", n);
```

```
    return 0;
}
```

Q5: bounded.c

```
/* bounded.c - Q5: bounded buffer with semaphores + mutex. usage: ./bounded producers consumers
[nomutex] */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define SIZE 5
#ifndef ITEMS
#define ITEMS 20                    /* items 1..20, sum = 210 */
#endif
static int buf[SIZE], in = 0, out = 0;
static sem_t empty, full; static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static int nomutex, P, C, next_item = 1, taken = 0; static long total = 0; static int consumed_count[ITEMS +
1];
static pthread_mutex_t meta = PTHREAD_MUTEX_INITIALIZER;   /* protects only the item numbering
/ counting, not the buffer */
static void lock(void){ if (!nomutex) pthread_mutex_lock(&mtx); }
static void unlock(void){ if (!nomutex) pthread_mutex_unlock(&mtx); }
static void *producer(void *a){
    for (;;) {
        pthread_mutex_lock(&meta); int item = next_item <= ITEMS ? next_item++ : 0;
pthread_mutex_unlock(&meta);
        if (!item) break;
        /* TODO 1 (completed) */
        sem_wait(&empty);          /* wait for a free slot */
        lock(); buf[in] = item; in = (in + 1) % SIZE; unlock();
        sem_post(&full);           /* announce a new item */
    }
    return NULL;
}
static void *consumer(void *a){
    for (;;) {
        pthread_mutex_lock(&meta); if (taken >= ITEMS) { pthread_mutex_unlock(&meta); break; }
taken++; pthread_mutex_unlock(&meta);
        /* TODO 2 (completed) */
        sem_wait(&full);           /* wait for an item */
        lock(); int item = buf[out]; out = (out + 1) % SIZE; unlock();
        sem_post(&empty);          /* free the slot */
        pthread_mutex_lock(&meta); total += item; consumed_count[item]++;
pthread_mutex_unlock(&meta);
    }
    return NULL;
}
int main(int argc, char *argv[]) {
    P = atoi(argv[1]); C = atoi(argv[2]); nomutex = argc > 3;
```

```
    pthread_t p[16], c[16];
    sem_init(&empty, 0, SIZE); sem_init(&full, 0, 0);
    for (int i = 0; i < P; i++) pthread_create(&p[i], NULL, producer, NULL);
    for (int i = 0; i < C; i++) pthread_create(&c[i], NULL, consumer, NULL);
    for (int i = 0; i < P; i++) pthread_join(p[i], NULL);
    for (int i = 0; i < C; i++) pthread_join(c[i], NULL);
    int once = 1; for (int i = 1; i <= ITEMS; i++) if (consumed_count[i] != 1) once = 0;
    printf("producers=%d consumers=%d%s: sum=%ld every item consumed exactly once: %s (expected
%ld)\n", P, C, nomutex ? " (NO MUTEX)" : "", total, once ? "yes" : "NO", (long)ITEMS*(ITEMS+1)/2);
    return 0;
}
```

Q6: banker.c

```
/* banker.c - Q6: Banker's algorithm (safety + resource-request) for the shared laboratory cluster
   resources: A = GPUs (10), B = simulation licences (5), C = fast-storage slots (7) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXP 1000
#define MAXR 12
static int n, m;
static int avail[MAXR], max[MAXP][MAXR], alloc[MAXP][MAXR], need[MAXP][MAXR];

/* safety algorithm: returns 1 and fills seq[] if a safe sequence exists */
static int is_safe(int *seq) {
    int work[MAXR]; static char finish[MAXP]; int count = 0, found;
    memcpy(work, avail, m * sizeof(int)); memset(finish, 0, n);
    do {                                    /* one sweep over all unfinished processes */
        found = 0;
        for (int i = 0; i < n; i++) {
            if (finish[i]) continue;
            int ok = 1;
            for (int r = 0; r < m; r++) if (need[i][r] > work[r]) { ok = 0; break; }
            if (ok) {                       /* process i can run to completion */
                for (int r = 0; r < m; r++) work[r] += alloc[i][r];
                finish[i] = 1; if (seq) seq[count] = i; count++; found = 1;
            }
        }
    } while (found);
    return count == n;
}
/* resource-request algorithm: 1 granted, 0 refused (reason printed) */
static int request(int p, const int *req) {
    for (int r = 0; r < m; r++) if (req[r] > need[p][r]) { printf("  refused: P%d exceeds its declared
maximum\n", p); return 0; }
    for (int r = 0; r < m; r++) if (req[r] > avail[r])   { printf("  refused: not enough available now, P%d must
wait\n", p); return 0; }
    for (int r = 0; r < m; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; need[p][r] -= req[r]; }   /* pretend */
```

```
    int seq[MAXP];
    if (is_safe(seq)) { printf("  granted: new state is safe, e.g. <"); for (int i = 0; i < n; i++) printf("P%d%s",
seq[i], i < n-1 ? ", " : ">\n"); return 1; }
    for (int r = 0; r < m; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; need[p][r] += req[r]; }   /* roll back */
    printf("  refused: granting would leave an UNSAFE state, P%d must wait\n", p); return 0;
}
static void load_textbook(void) {
    n = 5; m = 3;
    int A[5][3] = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}}, M[5][3] =
{{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}}, V[3] = {3,3,2};
    memcpy(avail, V, sizeof V);
    for (int i = 0; i < n; i++) for (int r = 0; r < m; r++) { alloc[i][r] = A[i][r]; max[i][r] = M[i][r]; need[i][r] =
M[i][r] - A[i][r]; }
}
static void print_state(void) {
    printf("  Process  Allocation  Max      Need\n");
    for (int i = 0; i < n; i++) printf("  P%-7d %d %d %d       %d %d %d    %d %d %d\n", i,
alloc[i][0],alloc[i][1],alloc[i][2], max[i][0],max[i][1],max[i][2], need[i][0],need[i][1],need[i][2]);
    printf("  Available: %d %d %d\n", avail[0], avail[1], avail[2]);
}
/* worst-case state: only the last process can finish first, then the one before it, ... */
static void load_chain(int np, int nr) {
    n = np; m = nr; memset(avail, 0, sizeof avail); memset(alloc, 0, sizeof alloc); memset(need, 0, sizeof
need); memset(max, 0, sizeof max);
    avail[m-1] = 1;
    for (int i = 0; i < n; i++) { alloc[i][m-1] = 1; need[i][m-1] = n - i; max[i][m-1] = n - i + 1; }
}
static void load_random(int np, int nr) {                /* safe-by-construction random state: everyone can
finish */
    n = np; m = nr; srand(7);
    for (int r = 0; r < m; r++) avail[r] = np;             /* plenty free */
    for (int i = 0; i < n; i++) for (int r = 0; r < m; r++) { alloc[i][r] = rand() % 3; need[i][r] = rand() % 4;
max[i][r] = alloc[i][r] + need[i][r]; }
}
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static double time_safety(int *safe) {
    int reps = 1; double t;
    for (;;) { t = now(); for (int k = 0; k < reps; k++) *safe = is_safe(NULL); t = now() - t; if (t > 0.2 || reps >
(1<<24)) break; reps *= 4; }
    return t / reps;
}
int main(void) {
    printf("== Part C: textbook state (A=GPUs 10, B=licences 5, C=storage 7)\n");
    load_textbook(); print_state();
    int seq[MAXP]; printf("  Safety algorithm: %s", is_safe(seq) ? "SAFE, sequence <" : "UNSAFE\n");
    for (int i = 0; i < n; i++) printf("P%d%s", seq[i], i < n-1 ? ", " : ">\n");
    int r1[3] = {1,0,2}, r4[3] = {3,3,0}, r0[3] = {0,2,0};
    printf("\nRequest 1: P1 asks (1,0,2)\n"); request(1, r1);
    printf("Request 2: P4 asks (3,3,0)\n"); request(4, r4);
```

```
    printf("Request 3: P0 asks (0,2,0)  [state now includes P1's grant]\n"); request(0, r0);
    printf("\nFor comparison, the same P0 request (0,2,0) in the ORIGINAL state:\n"); load_textbook();
request(0, r0);
    printf("\n== Part E: time of the safety algorithm (average of many runs)\n");
    printf("%-34s %8s %4s %14s\n", "state", "n", "m", "time per check");
    int ns[] = {10, 100, 1000}, safe;
    for (int k = 0; k < 3; k++) { load_chain(ns[k], 3); double t = time_safety(&safe); printf("%-34s %8d %4d
%11.2f us  safe=%d\n", "worst case (chain)", ns[k], 3, t*1e6, safe); }
    for (int k = 0; k < 3; k++) { load_random(ns[k], 3); double t = time_safety(&safe); printf("%-34s %8d
%4d %11.2f us  safe=%d\n", "typical (everyone can finish)", ns[k], 3, t*1e6, safe); }
    int ms[] = {3, 6, 12};
    for (int k = 0; k < 3; k++) { load_chain(1000, ms[k]); double t = time_safety(&safe); printf("%-34s %8d
%4d %11.2f us  safe=%d\n", "worst case, vary m", 1000, ms[k], t*1e6, safe); }
    return 0;
}
```

Appendix B. Source Code for the Measurement Programs

Q1: many.c (2,000 processes and memory per process)

```
/* many.c - Q1: start N sleeping processes, report creation time and memory per process */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>
#include <sys/wait.h>
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static long field(const char*path,const char*key){FILE*f=fopen(path,"r");if(!f)return -1;char l[256];long
v=-1;
  while(fgets(l,sizeof l,f)) if(!strncmp(l,key,strlen(key))){v=atol(l+strlen(key));break;} fclose(f);return v;}
int main(int argc,char**argv){
  int n=atoi(argv[1]); size_t mb=argc>2?atol(argv[2]):0;
  char*big=NULL; if(mb){big=malloc(mb<<20); memset(big,1,mb<<20);}   /* data the session shares via
copy-on-write */
  pid_t*pids=malloc(n*sizeof*pids); double t=now();
  for(int i=0;i<n;i++){pid_t p=fork(); if(p==0){pause();_exit(0);} pids[i]=p;}
  double created=now()-t; usleep(300000);
  double rss=0,pss=0,pte=0; char path[64];
  for(int i=0;i<n;i++){
    snprintf(path,sizeof path,"/proc/%d/smaps_rollup",pids[i]); long v=field(path,"Pss:"); if(v>0)pss+=v;
    snprintf(path,sizeof path,"/proc/%d/status",pids[i]); v=field(path,"VmRSS:"); if(v>0)rss+=v;
    v=field(path,"VmPTE:"); if(v>0)pte+=v;
  }
  printf("N=%d parent_data=%zuMB create_time=%.1f ms (%.1f us
each)\n",n,mb,created*1e3,created*1e6/n);
  printf("avg per process: RSS=%.0f kB  PSS=%.0f kB  page-tables(VmPTE)=%.0f
kB\n",rss/n,pss/n,pte/n);
  FILE*f=fopen("/proc/meminfo","r");char l[128];while(fgets(l,sizeof
l,f))if(!strncmp(l,"MemAvailable",12))fputs(l,stdout);fclose(f);
for(int i=0;i<n;i++)kill(pids[i],SIGKILL); while(wait(NULL)>0);
return 0;}
```

Q1: ctx.c (context-switch time)

```
/* ctx.c - cost of one context switch: two processes (or two threads) ping-pong over pipes on ONE cpu */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sched.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
```

```
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static int a[2],b[2]; static int R;
static void pin(void){cpu_set_t s;CPU_ZERO(&s);CPU_SET(0,&s);sched_setaffinity(0,sizeof s,&s);}
static void*echo(void*x){char c;pin();for(int i=0;i<R;i++){read(a[0],&c,1);write(b[1],&c,1);}return 0;}
int main(int argc,char**argv){
 R=argc>2?atoi(argv[2]):100000; int thread=argv[1][0]=='t'; char c='x';
 pipe(a);pipe(b);pin(); pthread_t th; pid_t p=0;
 if(thread) pthread_create(&th,0,echo,0); else if((p=fork())==0){echo(0);_exit(0);}
 double t=now();
 for(int i=0;i<R;i++){write(a[1],&c,1);read(b[0],&c,1);}
 double d=now()-t;
 if(thread)pthread_join(th,0); else waitpid(p,0,0);
 printf("%s: %d round trips = %d switches in %.1f ms -> %.2f us per switch (includes 2 pipe syscalls)\n",
  thread?"threads  ":"processes",R,2*R,d*1e3,d*1e6/(2*R));
 return 0;}
```

Q2: sched.py (all scheduling algorithms)

```
import json, collections
JOBS = [  # name, arrival, burst, priority(lower number = higher priority)
 ("J1 film",0,20,3), ("J2 thumb",1,2,1), ("J3 clip",2,8,4), ("J4 thumb",3,1,2), ("J5 clip",4,5,5)]
def finish(res, jobs):
    seg=[];
    for t,n in res:                    # unit timeline: (time, name or None)
        if seg and seg[-1][0]==n and seg[-1][2]==t: seg[-1][2]=t+1
        else: seg.append([n,t,t+1])
    comp={}; first={}
    for n,s,e in seg:
        if n and n not in first: first[n]=s
        if n: comp[n]=e
    rows=[]
    for n,a,b,p in jobs:
        ta=comp[n]-a; w=ta-b; r=first[n]-a; rows.append((n,a,b,p,comp[n],w,ta,r))
    cs=sum(1 for i in range(1,len(seg)) if seg[i][0] and seg[i-1][0] and seg[i][0]!=seg[i-1][0])
    return dict(seg=[s for s in seg if s[0]], rows=rows,
        avg_w=sum(r[5] for r in rows)/len(rows), avg_t=sum(r[6] for r in rows)/len(rows),
        avg_r=sum(r[7] for r in rows)/len(rows), cs=cs)
def sim(jobs, pick, preempt=True):
    rem={n:b for n,a,b,p in jobs}; info={n:(a,b,p) for n,a,b,p in jobs}
    t=0; cur=None; out=[]
    while any(rem.values()):
        ready=[n for n in rem if rem[n]>0 and info[n][0]<=t]
        if not ready: out.append((t,None)); t+=1; continue
        if cur is None or rem[cur]==0 or preempt: cur=pick(ready,info,rem,cur)
        out.append((t,cur)); rem[cur]-=1; t+=1
    return out
def fcfs(r,i,rem,c): return min(r,key=lambda n:(i[n][0],n))
def sjf(r,i,rem,c): return c if c and rem[c]>0 else min(r,key=lambda n:(i[n][1],i[n][0]))
def srtf(r,i,rem,c): return min(r,key=lambda n:(rem[n],i[n][0]))
```

```
def prio(r,i,rem,c): return c if c and rem[c]>0 else min(r,key=lambda n:(i[n][2],i[n][0]))
def run_np(jobs,pick):
    out=sim(jobs,pick,preempt=False); return finish(out,jobs)
def rr(jobs,q):
    rem={n:b for n,a,b,p in jobs}; arr=sorted(jobs,key=lambda j:j[1]); idx=0; t=0; Q=collections.deque();
out=[]
    def admit(t):
        nonlocal idx
        while idx<len(arr) and arr[idx][1]<=t: Q.append(arr[idx][0]); idx+=1
    admit(0)
    while any(rem.values()):
        if not Q: out.append((t,None)); t+=1; admit(t); continue
        n=Q.popleft(); run=min(q,rem[n])
        for _ in range(run): out.append((t,n)); t+=1; rem[n]-=1; admit(t)
        if rem[n]>0: Q.append(n)
    return finish(out,jobs)
def mlfq(jobs,quanta=(2,6,10**9)):
    rem={n:b for n,a,b,p in jobs}; arr={n:a for n,a,b,p in jobs}
    Q=[collections.deque() for _ in quanta]; level={}; used=0; cur=None; t=0; out=[]; added=set()
    while any(rem.values()):
        for n in rem:
            if n not in added and arr[n]<=t: Q[0].append(n); level[n]=0; added.add(n)
        # highest nonempty queue
        top=next((l for l in range(len(Q)) if Q[l]),None)
        if cur is not None and top is not None and top<level[cur]:   # preempted by higher queue
            Q[level[cur]].appendleft(cur) if False else Q[level[cur]].append(cur); cur=None
        if cur is None:
            if top is None: out.append((t,None)); t+=1; continue
            cur=Q[top].popleft(); used=0
        out.append((t,cur)); rem[cur]-=1; used+=1; t+=1
        if rem[cur]==0: cur=None
        elif used>=quanta[level[cur]]:
            level[cur]=min(level[cur]+1,len(Q)-1); Q[level[cur]].append(cur); cur=None
    return finish(out,jobs)
def show(title,r):
    print(f"== {title}")
    print(" Gantt:", " | ".join(f"{n.split()[0]}[{s}-{e}]" for n,s,e in r['seg']))
    for row in r['rows']: print("  %-9s arr=%2d burst=%2d prio=%d  done=%2d  wait=%2d  turn=%2d
resp=%2d"%row)
    print("  AVG wait=%.2f turnaround=%.2f response=%.2f  context
switches=%d"%(r['avg_w'],r['avg_t'],r['avg_r'],r['cs']))
R={}
R['FCFS']=run_np(JOBS,fcfs); R['SJF']=run_np(JOBS,sjf)
R['SRTF']=finish(sim(JOBS,srtf,True),JOBS); R['Priority']=run_np(JOBS,prio)
for q in (1,3,5,20): R[f'RR q={q}']=rr(JOBS,q)
R['MLFQ']=mlfq(JOBS)
for k,v in R.items(): show(k,v)
short=lambda r:[x for x in r['rows'] if x[2]<=2]; longj=lambda r:[x for x in r['rows'] if x[2]>=8]
print("\n== short(burst<=2) vs long(burst>=8) average waiting")
for k in ['FCFS','SJF','SRTF','Priority','RR q=3','RR q=5','MLFQ']:
```

```
    s=short(R[k]); l=longj(R[k]); print("%-9s short=%.2f long=%.2f"%(k,sum(x[5] for x in
s)/len(s),sum(x[5] for x in l)/len(l)))
json.dump({k:dict(seg=v['seg'],avg_w=v['avg_w'],avg_t=v['avg_t'],avg_r=v['avg_r'],cs=v['cs'],rows=v['row
s']) for k,v in R.items()},open('sched.json','w'))
# starvation + ageing
def starve(age):
    # J0 runs 0-3, L (prio 9, burst 3) arrives t=1; a new prio-1 job (burst 2) arrives at every even time 2..60
    jobs=[("J0",0,3,1),("L low",1,3,9)]+[(f"H{t}",t,2,1) for t in range(2,61,2)]
    rem={n:b for n,a,b,p in jobs}; info={n:(a,b,p) for n,a,b,p in jobs}; t=0; start={}
    while any(rem.values()):
        ready=[n for n in rem if rem[n]>0 and info[n][0]<=t]
        if not ready: t+=1; continue
        eff=lambda n: info[n][2]-(((t-info[n][0])//age) if age else 0)
        n=min(ready,key=lambda n:(eff(n),info[n][0])); start.setdefault(n,t); t+=rem[n]; rem[n]=0
    return start["L low"]
print("\nstarvation: low-priority job starts at t=%d without ageing, t=%d with ageing (+1 priority per 5
units waiting)"%(starve(0),starve(5)))
```

Q3: threads.c

```
/* threads.c - Q3: process 64 "camera frames" sequentially, with a thread per camera, and with pools */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <stdatomic.h>
#include <time.h>
#include <sys/wait.h>
#define CAMERAS 64
static long WORK; static int IO_US;
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static unsigned long process_frame(int cam){        /* stand-in for plate reading: fixed CPU work + camera
wait */
    if (IO_US) usleep(IO_US);
    unsigned long x = cam * 2654435761UL + 1;
    for (long i = 0; i < WORK; i++) x = x * 6364136223846793005UL + 1442695040888963407UL;
    return x;
}
static atomic_int next_cam; static atomic_ulong checksum; static atomic_int processed;
static void *pool_worker(void *a){ int c; while ((c = atomic_fetch_add(&next_cam,1)) < CAMERAS){
atomic_fetch_add(&checksum,process_frame(c)); atomic_fetch_add(&processed,1);} return 0; }
static void *cam_thread(void *a){ atomic_fetch_add(&checksum,process_frame((int)(long)a));
atomic_fetch_add(&processed,1); return 0; }
static double run_threads(int k){                   /* k==0: one thread per camera */
    atomic_store(&next_cam,0); atomic_store(&checksum,0); atomic_store(&processed,0);
    int n = k ? k : CAMERAS; pthread_t th[CAMERAS]; double t = now();
    for (int i = 0; i < n; i++) k ? pthread_create(&th[i],0,pool_worker,0) :
pthread_create(&th[i],0,cam_thread,(void*)(long)i);
```

```
    for (int i = 0; i < n; i++) pthread_join(th[i],0);
    return now()-t;
}
static double run_procs(int k){                    /* k processes, results combined through pipes */
    int fd[16][2]; double t = now(); pid_t p[16];
    for (int i = 0; i < k; i++){ pipe(fd[i]); if ((p[i]=fork())==0){ unsigned long s=0; for(int
c=i;c<CAMERAS;c+=k) s+=process_frame(c); write(fd[i][1],&s,sizeof s); _exit(0);} }
    unsigned long total=0,v; for (int i=0;i<k;i++){ read(fd[i][0],&v,sizeof v); total+=v; waitpid(p[i],0,0);}
    return now()-t;
}
static volatile long plain; static long locked; static atomic_long atom; static pthread_mutex_t m =
PTHREAD_MUTEX_INITIALIZER;
static void *inc(void *a){ for (long i=0;i<5000000;i++){ plain++; pthread_mutex_lock(&m); locked++;
pthread_mutex_unlock(&m); atomic_fetch_add(&atom,1);} return 0; }
int main(int argc,char**argv){
    IO_US = argc>1 ? atoi(argv[1]) : 0;
    int io=IO_US; IO_US=0; WORK=10000000; double t=now(); volatile unsigned long
sink=process_frame(1); (void)sink; double per10m=now()-t;
    WORK=(long)(10000000*0.020/per10m); IO_US=io; fprintf(stderr,"WORK=%ld\n",WORK);  /*
calibrate: about 20 ms of CPU per frame */
    t=now(); unsigned long cs=0; for(int c=0;c<CAMERAS;c++) cs+=process_frame(c); double seq=now()-
t;
    printf("frames=%d io_wait=%d us/frame cpu~20ms/frame\n",CAMERAS,IO_US);
    printf("sequential           : %.3f s  speed-up 1.00\n",seq);
    double d=run_threads(0); printf("thread per camera(64): %.3f s  speed-up %.2f  checksum
ok=%d\n",d,seq/d,(int)(atomic_load(&checksum)==cs));
    int ks[]={4,8,16};
    for(int i=0;i<3;i++){ d=run_threads(ks[i]); printf("pool of %2d threads   : %.3f s  speed-up %.2f
frames=%d checksum
ok=%d\n",ks[i],d,seq/d,atomic_load(&processed),(int)(atomic_load(&checksum)==cs)); }
    for(int i=0;i<3;i++){ int k=ks[i]>8?8:ks[i]; d=run_procs(k); printf("pool of %2d processes : %.3f s
speed-up %.2f\n",k,d,seq/d); }
    if (!IO_US){ pthread_t a,b; pthread_create(&a,0,inc,0); pthread_create(&b,0,inc,0); pthread_join(a,0);
pthread_join(b,0);
      printf("shared counter, 2 threads x 5,000,000 increments (expected 10000000): unprotected=%ld
mutex=%ld atomic=%ld\n",plain,locked,(long)atomic_load(&atom)); }
    return 0; }
```

Q4: ipc_bench.c

```
/* ipc_bench.c - Q4: time to move N MB from one process to another: pipe, POSIX message queue, shared
memory + semaphores */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <semaphore.h>
#include <mqueue.h>
```

```
#include <sys/mman.h>
#include <sys/wait.h>
#include <time.h>
#define CH 65536
#define SLOTS 16
#define MQ_CH 8192
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static void fill(unsigned long *b,size_t n,unsigned long s){for(size_t i=0;i<n/8;i++)b[i]=s+i;}      /*
producer creates data */
static unsigned long sum(const unsigned long *b,size_t n){unsigned long s=0;for(size_t
i=0;i<n/8;i++)s+=b[i];return s;} /* consumer uses data */
static unsigned long expect(size_t total,size_t ch){unsigned long s=0;for(size_t
o=0;o<total;o+=ch){unsigned long c=o/ch;for(size_t i=0;i<ch/8;i++)s+=c+i;}return s;}
static double t_pipe(size_t total){
  int fd[2];pipe(fd);double t=now();pid_t p=fork();
  if(p==0){close(fd[1]);unsigned long*b=malloc(CH),s=0;ssize_t n;size_t got=0;
    while((n=read(fd[0],b,CH))>0){ssize_t g=n;while(g<CH&&g>0){ssize_t m=read(fd[0],(char*)b+g,CH-
g);if(m<=0)break;g+=m;} s+=sum(b,CH);got+=g;}
    _exit(got==total&&s==expect(total,CH)?0:1);}
  close(fd[0]);unsigned long*b=malloc(CH);
  for(size_t o=0;o<total;o+=CH){fill(b,CH,o/CH);size_t w=0;while(w<CH){ssize_t
m=write(fd[1],(char*)b+w,CH-w);w+=m;}}
  close(fd[1]);int st;waitpid(p,&st,0);double d=now()-t;if(WEXITSTATUS(st))fprintf(stderr,"pipe: DATA
ERROR\n");return d;}
struct shm{sem_t empty,full;char slot[SLOTS][CH];};
static double t_shm(size_t total){
  struct shm*m=mmap(0,sizeof*m,PROT_READ|PROT_WRITE,MAP_SHARED|MAP_ANONYMOUS,-
1,0);
  sem_init(&m->empty,1,SLOTS);sem_init(&m->full,1,0);size_t chunks=total/CH;double t=now();pid_t
p=fork();
  if(p==0){unsigned long s=0;for(size_t c=0;c<chunks;c++){sem_wait(&m->full);s+=sum((unsigned
long*)m->slot[c%SLOTS],CH);sem_post(&m->empty);}
    _exit(s==expect(total,CH)?0:1);}
  for(size_t  c=0;c<chunks;c++){sem_wait(&m->empty);fill((unsigned  long*)m-
>slot[c%SLOTS],CH,c);sem_post(&m->full);}
  int st;waitpid(p,&st,0);double d=now()-t;if(WEXITSTATUS(st))fprintf(stderr,"shm: DATA
ERROR\n");munmap(m,sizeof*m);return d;}
static double t_mq(size_t total){
  struct mq_attr a={0,10,MQ_CH,0};mq_unlink("/ipcb");mqd_t
q=mq_open("/ipcb",O_CREAT|O_RDWR,0600,&a);if(q<0){perror("mq_open");return -1;}
double t=now();pid_t p=fork();size_t chunks=total/MQ_CH;
  if(p==0){unsigned long*b=malloc(MQ_CH),s=0;for(size_t
c=0;c<chunks;c++){mq_receive(q,(char*)b,MQ_CH,NULL);s+=sum(b,MQ_CH);}_exit(s==expect(total,
MQ_CH)?0:1);}
  unsigned long*b=malloc(MQ_CH);for(size_t
c=0;c<chunks;c++){fill(b,MQ_CH,c);mq_send(q,(char*)b,MQ_CH,0);}
  int st;waitpid(p,&st,0);double d=now()-t;if(WEXITSTATUS(st))fprintf(stderr,"mq: DATA
ERROR\n");mq_close(q);mq_unlink("/ipcb");return d;}
int main(void){
```

```
 size_t mbs[]={1,10,100};
 printf("%-8s %12s %12s %12s  (best of 3, seconds; data verified by checksum)\n","size","pipe","msg queue","shared mem");
 for(int i=0;i<3;i++){size_t tot=mbs[i]<<20;double bp=1e9,bq=1e9,bs=1e9;
  for(int r=0;r<3;r++){double x;x=t_pipe(tot);if(x<bp)bp=x;x=t_mq(tot);if(x<bq)bq=x;x=t_shm(tot);if(x<bs)bs=x;}
  printf("%-4zu MB  %12.4f %12.4f %12.4f  => MB/s: %6.0f %6.0f %6.0f\n",mbs[i],bp,bq,bs,mbs[i]/bp,mbs[i]/bq,mbs[i]/bs);}
 return 0;}
```

Q5: peterson.c

```
/* peterson.c - Peterson's algorithm for 2 threads: plain variables vs sequentially consistent atomics */
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>
#define N 5000000
static volatile int pflag[2], pturn; static atomic_int aflag[2], aturn;
static volatile long shared_plain, shared_atomic;   /* volatile: stop the compiler caching it in a register */
static void *plain(void *a){ int i=(int)(long)a, j=1-i;
 for (int n=0;n<N;n++){ pflag[i]=1; pturn=j; while (pflag[j] && pturn==j) ; shared_plain++; pflag[i]=0; }
return 0; }
static void *seqcst(void *a){ int i=(int)(long)a, j=1-i;
 for (int n=0;n<N;n++){ atomic_store(&aflag[i],1); atomic_store(&aturn,j); while (atomic_load(&aflag[j])
&& atomic_load(&aturn)==j) ; shared_atomic++; atomic_store(&aflag[i],0); } return 0; }
int main(void){ pthread_t a,b;
 pthread_create(&a,0,plain,(void*)0); pthread_create(&b,0,plain,(void*)1); pthread_join(a,0);
pthread_join(b,0);
 printf("Peterson, plain volatile variables : counter=%ld expected=%d lost updates=%ld\n",shared_plain,2*N,2*N-shared_plain);
 pthread_create(&a,0,seqcst,(void*)0); pthread_create(&b,0,seqcst,(void*)1); pthread_join(a,0);
pthread_join(b,0);
 printf("Peterson, seq_cst atomics (fences) : counter=%ld expected=%d lost updates=%ld\n",shared_atomic,2*N,2*N-shared_atomic); return 0; }
```

Q5: dining.c

```
/* dining.c - 5 philosophers; "room" semaphore (max 4 seated) prevents deadlock. ./dining [naive] */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#define P 5
#define MEALS 2000
static sem_t fork_[P], room; static int naive; static long meals[P];
static void *phil(void *a){ int i=(int)(long)a;
 for (int m=0;m<MEALS;m++){
  if (!naive) sem_wait(&room);
  sem_wait(&fork_[i]); if (naive) usleep(100); sem_wait(&fork_[(i+1)%P]);   /* pick left, pause, pick right */
```

```
    meals[i]++;
    sem_post(&fork_[(i+1)%P]); sem_post(&fork_[i]);
    if (!naive) sem_post(&room);
  } return 0; }
int main(int argc,char**argv){ naive=argc>1; pthread_t t[P]; sem_init(&room,0,P-1);
  for(int i=0;i<P;i++)sem_init(&fork_[i],0,1);
  for(long i=0;i<P;i++)pthread_create(&t[i],0,phil,(void*)i);
  for(int i=0;i<P;i++)pthread_join(t[i],0);
  long s=0;for(int i=0;i<P;i++)s+=meals[i]; printf("%s: all philosophers finished, meals eaten = %ld
(expected %d)\n",naive?"naive":"with room semaphore",s,P*MEALS); return 0; }
```

Q5: booking.c

```
/* booking.c - Q5: seat booking with no lock, one global lock, one lock per train, one lock per coach */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#define TRAINS 20
#define COACHES 10
#define SEATS 72
#define REQ 20000                       /* booking requests per thread */
static unsigned char booked[TRAINS][COACHES][SEATS];
static pthread_mutex_t g = PTHREAD_MUTEX_INITIALIZER, lt[TRAINS], lc[TRAINS][COACHES];
static int mode, work_iters;            /* 0 none, 1 global, 2 train, 3 coach */
static long sold[64];
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static void spin(int n){ for (int i = 0; i < n; i++) asm volatile("" ::: "memory"); }  /* stand-in for checking
payment/fare tables */
static void *agent(void *arg){
    unsigned s = (unsigned)(long)arg * 2654435761u + 12345; long ok = 0;
    for (int r = 0; r < REQ; r++) {
        s = s * 1103515245u + 12345; int t = (s >> 8) % TRAINS;
        s = s * 1103515245u + 12345; int c = (s >> 8) % COACHES;
        s = s * 1103515245u + 12345; int k = (s >> 8) % SEATS;
        pthread_mutex_t *m = mode == 1 ? &g : mode == 2 ? &lt[t] : mode == 3 ? &lc[t][c] : NULL;
        if (m) pthread_mutex_lock(m);
        int is_free = !booked[t][c][k];         /* check ... */
        spin(work_iters);                       /* ... fare and payment work ... */
        if (is_free) { booked[t][c][k] = 1; ok++; }     /* ... then act */
        if (m) pthread_mutex_unlock(m);
    }
    sold[(long)arg] = ok; return NULL;
}
int main(int argc, char **argv){
    work_iters = argc > 1 ? atoi(argv[1]) : 500;
    for (int t = 0; t < TRAINS; t++){ pthread_mutex_init(&lt[t],0); for (int c = 0; c < COACHES; c++)
pthread_mutex_init(&lc[t][c],0); }
```

```
   const char *names[] = {"no lock","one global lock","one lock per train","one lock per coach"};
   printf("%d trains x %d coaches x %d seats = %d seats; %d requests per
thread\n",TRAINS,COACHES,SEATS,TRAINS*COACHES*SEATS,REQ);
   printf("%-20s %7s %14s %12s %10s\n","locking","threads","requests/s","tickets sold","double-sold");
   int tc[] = {1,2,4,8};
   for (mode = 0; mode < 4; mode++) for (int i = 0; i < 4; i++){
      int n = tc[i]; double rates[7]; long tot=0, dbl=0;
      for (int rep = 0; rep < 7; rep++){
         for (int t=0;t<TRAINS;t++)for(int c=0;c<COACHES;c++)for(int
k=0;k<SEATS;k++)booked[t][c][k]=0;
         pthread_t th[8]; double t0 = now();
         for (long j = 0; j < n; j++) pthread_create(&th[j],0,agent,(void*)j);
         for (int j = 0; j < n; j++) pthread_join(th[j],0);
         double d = now()-t0; long sum = 0, seats = 0; for (int j=0;j<n;j++) sum += sold[j];
         for (int t=0;t<TRAINS;t++)for(int c=0;c<COACHES;c++)for(int
k=0;k<SEATS;k++)seats+=booked[t][c][k];
         rates[rep]=(double)n*REQ/d; tot=sum; dbl+=sum-seats;
      }
      for (int x=0;x<7;x++)for(int y=x+1;y<7;y++)if(rates[y]<rates[x]){double
tmp=rates[x];rates[x]=rates[y];rates[y]=tmp;}
      printf("%-20s %7d %14.0f %12ld %10ld\n",names[mode],n,rates[3],tot,dbl);   /* median of 7 runs;
double-sold summed over 7 runs */
   }
   return 0; }
```

Q5: counter.c

```
/* counter.c - Q5 alternative: shared counter protected by atomic, spin lock, mutex, semaphore */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdatomic.h>
#include <time.h>
#include <sys/resource.h>
static atomic_long a_cnt; static long cnt; static atomic_flag spin = ATOMIC_FLAG_INIT;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; static sem_t s;
static int method; static long OPS; static int CS_NS;
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static void hold(void){ if (CS_NS) { double t=now(); while ((now()-t)*1e9 < CS_NS) ; } }   /* long critical
section: busy for CS_NS */
static void *worker(void *a){
   for (long i = 0; i < OPS; i++) switch (method) {
      case 0: atomic_fetch_add(&a_cnt,1); break;
      case 1: while (atomic_flag_test_and_set_explicit(&spin,memory_order_acquire)) ; cnt++; hold();
atomic_flag_clear_explicit(&spin,memory_order_release); break;
      case 2: pthread_mutex_lock(&m); cnt++; hold(); pthread_mutex_unlock(&m); break;
      case 3: sem_wait(&s); cnt++; hold(); sem_post(&s); break; }
```

```
    return 0; }
static double cpu(void){ struct rusage r; getrusage(RUSAGE_SELF,&r); return
r.ru_utime.tv_sec+r.ru_utime.tv_usec/1e6+r.ru_stime.tv_sec+r.ru_stime.tv_usec/1e6; }
int main(int argc,char**argv){
    CS_NS = argc>1?atoi(argv[1]):0; long ops_total = argc>2?atol(argv[2]):4000000; sem_init(&s,0,1);
    const char*nm[]={"atomic_fetch_add","spin lock","mutex","semaphore"}; int th[]={1,4,16};
    printf("critical section = %d ns, %ld updates in total per run\n",CS_NS,ops_total);
    printf("%-17s %7s %10s %14s %12s %8s\n","method","threads","wall (s)","ns per update","CPU time
(s)","correct");
    for (method=0; method<4; method++) for (int i=0;i<3;i++){
        int n=th[i]; OPS=ops_total/n; a_cnt=0; cnt=0; pthread_t t[16]; double c0=cpu(), w0=now();
        for(int j=0;j<n;j++)pthread_create(&t[j],0,worker,0); for(int j=0;j<n;j++)pthread_join(t[j],0);
        double w=now()-w0, c=cpu()-c0; long got = method==0?(long)atomic_load(&a_cnt):cnt;
        printf("%-17s %7d %10.3f %14.1f %12.3f
%8s\n",nm[method],n,w,w*1e9/(OPS*n),c,got==OPS*n?"yes":"NO"); }
    return 0; }
```

Sample output:

Q2.

```
!gcc -O2 -o rr rr.c
!./rr 3
```

```
Round Robin, quantum = 3
Gantt: J1[0-3] J2[3-5] J3[5-8] J4[8-9] J1[9-12] J5[12-15] J3[15-18] J1[18-21] J5[21-23] J3[23-25] J1[25-28] J1[28-31] J1[31-34] J1[34-36]

Job  Arr Burst Done Wait Turn Resp
J1     0    20   36   16   36    0
J2     1     2    5    2    4    2
J3     2     8   25   15   23    3
J4     3     1    9    5    6    5
J5     4     5   23   14   19    8
Average waiting = 10.40  turnaround = 17.60  response = 3.60  context switches = 10
```

Q4.

```
!gcc -O2 -o pipe_demo pipe_demo.c
!./pipe_demo 1000
```

```
cc1: fatal error: pipe_demo.c: No such file or directory
compilation terminated.
/bin/bash: line 1: ./pipe_demo: No such file or directory
```

Q5.

```
Writing bounded.c
```

```
!gcc -O2 -pthread -o bounded bounded.c
!./bounded 1 1
!./bounded 4 4
```

```
producers=1 consumers=1: sum=210 every item consumed exactly once: yes (expected 210)
producers=4 consumers=4: sum=210 every item consumed exactly once: yes (expected 210)
```

Q6.

```
!gcc -O2 -o banker banker.c
!./banker
```

```
== Part C: textbook state (A=GPUs 10, B=licences 5, C=storage 7)
  Process  Allocation  Max      Need
  P0       0 1 0       7 5 3    7 4 3
  P1       2 0 0       3 2 2    1 2 2
  P2       3 0 2       9 0 2    6 0 0
  P3       2 1 1       2 2 2    0 1 1
  P4       0 0 2       4 3 3    4 3 1
  Available: 3 3 2
  Safety algorithm: SAFE, sequence <P1, P3, P4, P0, P2>

Request 1: P1 asks (1,0,2)
  granted: new state is safe, e.g. <P1, P3, P4, P0, P2>
Request 2: P4 asks (3,3,0)
  refused: not enough available now, P4 must wait
Request 3: P0 asks (0,2,0)  [state now includes P1's grant]
  refused: granting would leave an UNSAFE state, P0 must wait

For comparison, the same P0 request (0,2,0) in the ORIGINAL state:
  granted: new state is safe, e.g. <P3, P1, P2, P0, P4>

== Part E: time of the safety algorithm (average of many runs)
state                                    n    m time per check
worst case (chain)                      10    3        0.26 us  safe=1
worst case (chain)                     100    3       22.43 us  safe=1
worst case (chain)                    1000    3     1896.72 us  safe=1
typical (everyone can finish)           10    3        0.08 us  safe=1
typical (everyone can finish)          100    3        0.72 us  safe=1
typical (everyone can finish)         1000    3        9.22 us  safe=1
worst case, vary m                    1000    3     4154.27 us  safe=1
worst case, vary m                    1000    6     5770.36 us  safe=1
worst case, vary m                    1000   12    10506.28 us  safe=1
```